University of St.Gallen

Principled Architectural Design for Distributed Information Systems



Simon Mayer, Alessandro Giugno, Luka Bekavac

University of St.Gallen
Master in Computer Science

Advanced Software and Systems Engineering (7,856)

PDF version of 11.11.2026 (the online slides may be newer)

From insight to impact.

Syllabus

Lecture

Exercise

Assignment

14.09.2026	Fundamentals of Software Architecture	Event Storming, Architecture Characteristics, ADRs, Service Granularity, Spring 101
21.09.2026	Hard Parts I: Service Granularity	
28.09.2026	Performance Testing and Fitness Functions	
05.10.2026	Hard Parts II: Code Sharing	Hexagonal Architecture, Persistence, Interactions, Testing
12.10.2026	Hard Parts III: Data Ownership and Access	
19.10.2026	Intermediate Presentations	
09.11.2026	Principled Architectural Design for Distributed Information Systems	Uniform Interaction, Event-Driven Interaction, Hypermedia Ecosystems
16.11.2026	Interaction Styles	
23.11.2026	Decoupling and Interoperability	
30.11.2026	Hypermedia APIs	Semantic Hypermedia, Agents and the Web of Things
07.12.2026	Interaction with AI Agents and with Physical Devices	
14.12.2026	Final Presentations	

Lecture Learning Goals

Lecture 7: Understand how to **select fitting architectural constraints** to achieve a desired non-functional profile
Assignment 7: Uniform Interaction? Exploring the REST Architectural Style

Lecture 8: Understand how to **select fitting interaction styles** to achieve a desired non-functional profile
Assignment 8: What Interaction Style? Event-based Interaction using MQTT and W3C WebSub

Lecture 9: Understand **interactive ecosystems** and how data can be separated from applications
Assignment 9: Who Owns the Data? Data Sovereignty with Solid Pods and WebIDs



Plugfest with Pitas (Nov 24, 5 to 8pm)

Lecture 10: Understand that **decoupling may happen on many levels** beyond the immediate API
Assignment 10: Multi-level Decoupling? Decoupling through Semantic Hypermedia

Lecture 11: Understand **what changes** when higher autonomy potential and constrained devices enter the game
Assignment 11: Autonomy and Interactions with Constrained Devices

Recap Lecture 7a: Overall Goal: Conscious Handling of Architectural Tr... ↗

Principled Architectural Design

As the **architect of a (distributed) (information) system**, you should be aware of the **design trade-offs that are induced by specific choices** in order to construct a system **with your desired non-functional properties.**

This is relevant in a (technology) world with projects that **exhibit a broad range of non-functional requirements.**

A desired non-functional profile (i.e., a set of non-functional properties) can be retained by **making specific coordinated architectural decisions** that **restrict individual architectural elements and their interfaces and relationships**

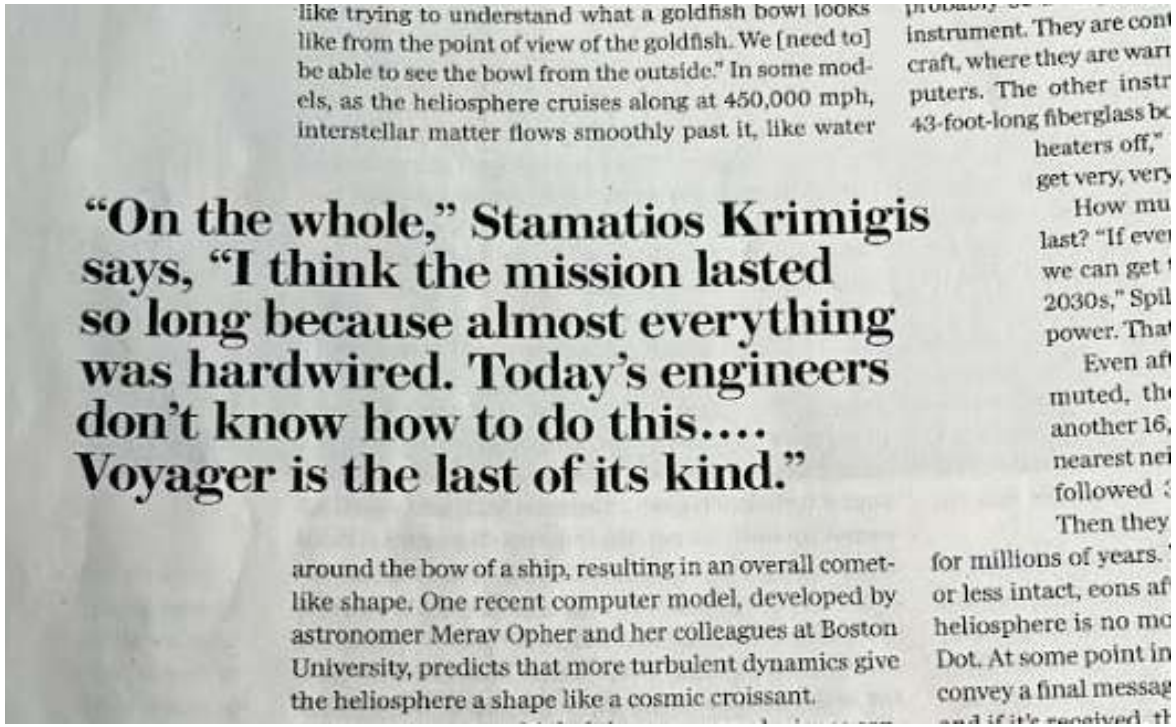


Architectural styles are abstractions that encapsulate these decisions
They provide guidance towards positioning a system at a **specific non-functional trade-off**

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Let's Talk NFPs




like trying to understand what a goldfish bowl looks like from the point of view of the goldfish. We [need to] be able to see the bowl from the outside." In some models, as the heliosphere cruises along at 450,000 mph, interstellar matter flows smoothly past it, like water

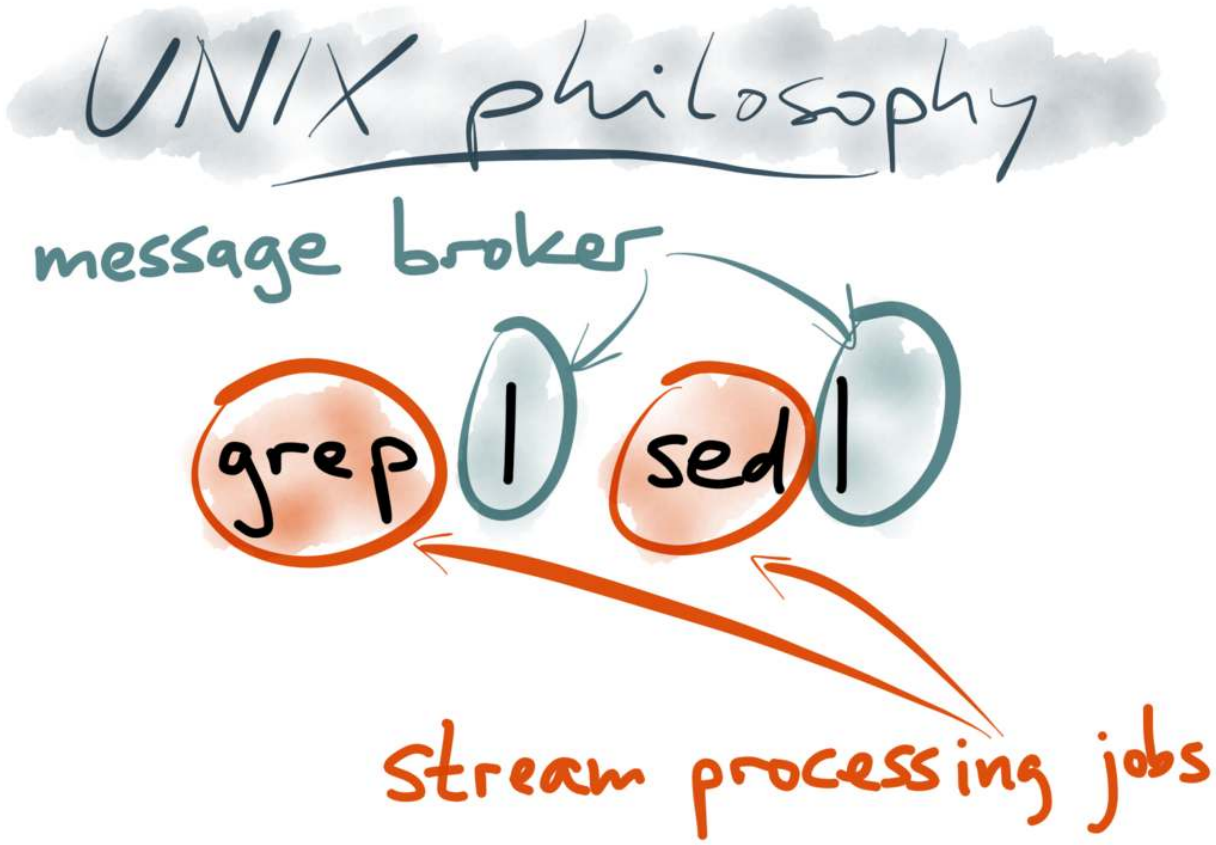
"On the whole," Stamatios Krimigis says, "I think the mission lasted so long because almost everything was hardwired. Today's engineers don't know how to do this.... Voyager is the last of its kind."

around the bow of a ship, resulting in an overall comet-like shape. One recent computer model, developed by astronomer Merav Opher and her colleagues at Boston University, predicts that more turbulent dynamics give the heliosphere a shape like a cosmic croissant.

instrument. They are cont
craft, where they are warr
puters. The other instr
43-foot-long fiberglass bo
heaters off,"
get very, very
How mu
last? "If ever
we can get
2030s," Spil
power. That
Even aft
muted, the
another 16,
nearest nei
followed
Then they
for millions of years.
or less intact, eons af
heliosphere is no mo
Dot. At some point in
convey a final messag

Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

Let's Talk NFPs



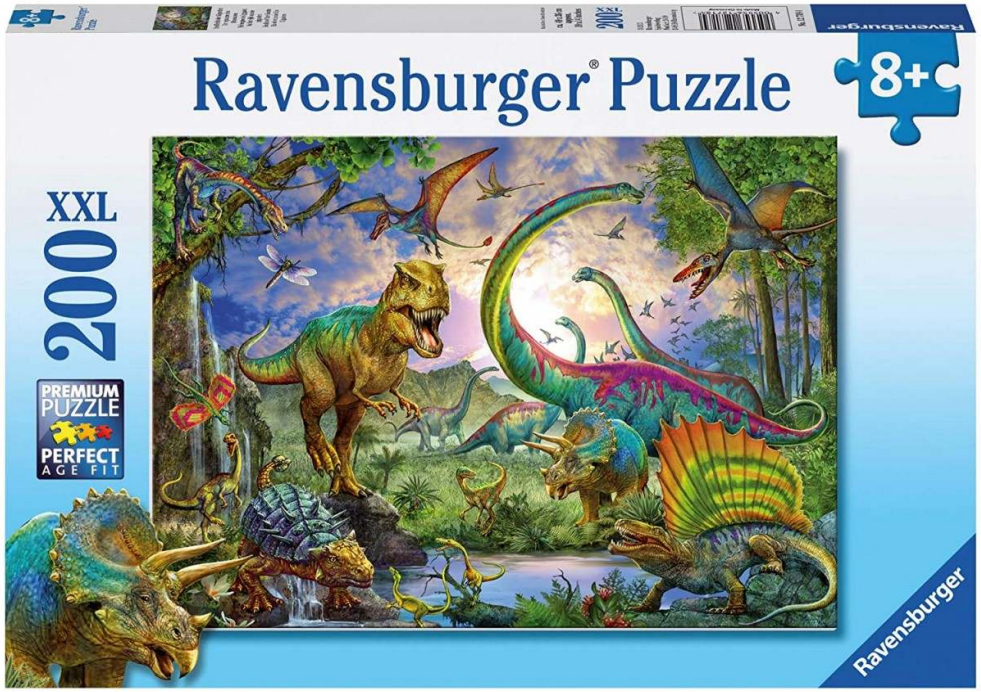
Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

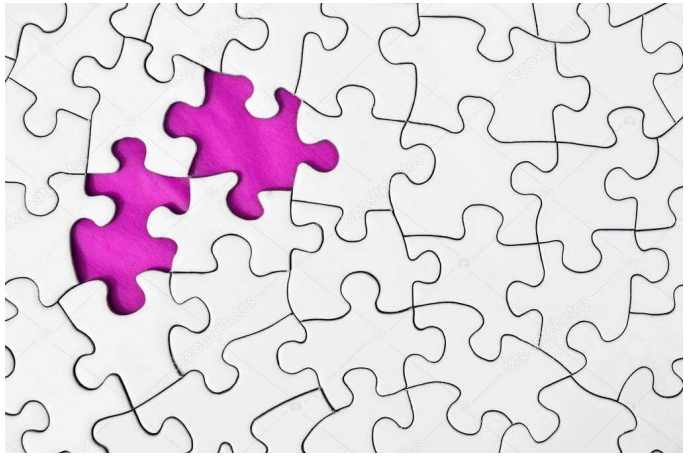
Let’s Talk NFPs



Non-Functional Profile
Interoperability
Responsiveness
Scalability
Extensibility
Autonomy
Testability
Consistency
Adaptability
Mobility
…
…
…

Let's Talk NFPs





Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

Let’s Talk NFPs

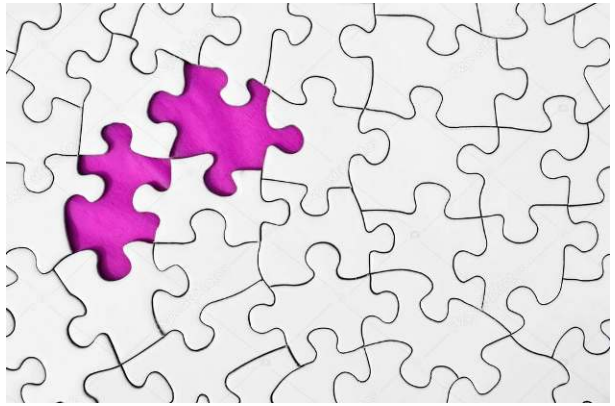




Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

Case Study 1: Creative Play





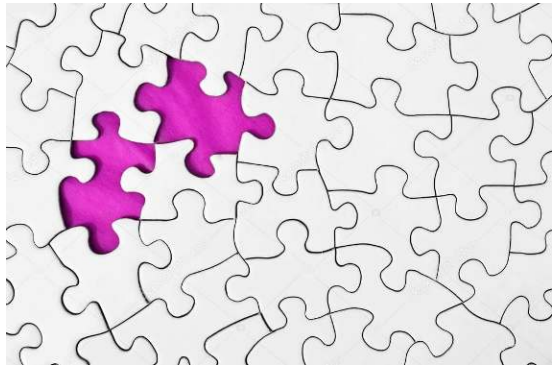
What are the defining features of **a single Lego brick**?

What are the defining features of **a single puzzle piece**?

For each, what happens if they are combined to **larger systems**?

Play and Discuss!





What are the **interfaces** of your individual bricks/pieces?

In what sense are they **uniform**?

What non-functional aspects do **uniform interfaces contribute** to?

What non-functional aspects do they tend to **undermine**?

How can we **make non-uniform interfaces uniform**?

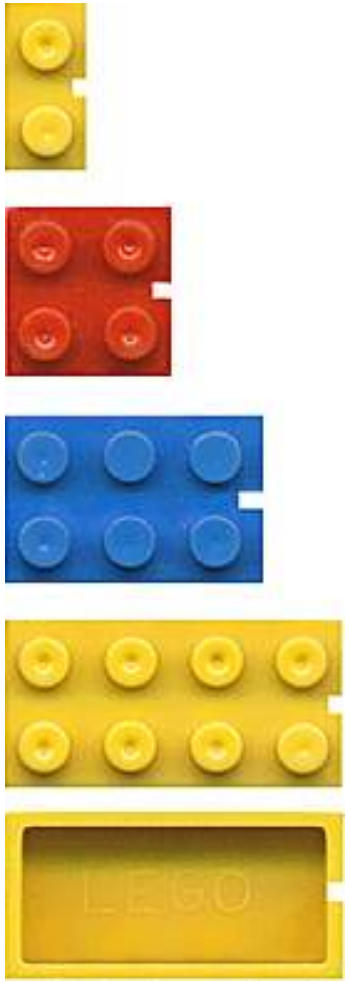
Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

Remember: 'Uniform Interface' in CS refers to a principle in the REST architectural style. It means that individual services are accessed uniformly, no matter what the services do. A prominent example are HTTP methods that apply uniformly to any Web resource.

Case Study 1: Creative Play

Ole Christiansen‘s „Leg godt“ („Play Well“) in 1932
Godtfred Christiansen‘s „system for creative play“ in 1958
Derivative of Kiddicraft play system
Lego was designed **with the system in mind**





Main desired (non-functional) characteristics: **Many recombinations in sturdy designs!**

Problem #1: **Light but durable materials**!

- Hollow bricks are desired
- However, especially the notches need to be resistant to impact and heat
- Solution: Injection-molded acrylonitrile butadiene styrene (ABS)



Problem #2: **Interlocks need to be sturdier** than for previous blocks!

Case Study 1: Creative Play

Christiansen‘s „Automatic Binding Brick“ (1958; later: Lego brick)

The relevant difference to competitors is on the system level – it is the interfaces!

Why is this so **important**?

Toy elements of this kind will be referred to generally as building bricks, and the principal object of the invention is to provide improved coupling means for clamping such building bricks together in any desired relative position thus providing for a vast variety of combinations of the bricks for making toy structures of many different kinds and shapes.

Fundamentally, a toy building element according to the invention comprises a base plate provided with pro-

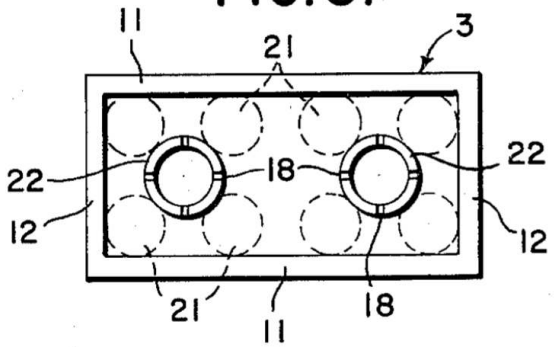
having four, six, and eight primary projections, although the invention is naturally not limited to the use of such

large number of primary projections **21**. Bricks provided with primary projections arranged in this manner are well-known per se.

It is also well-known to provide hollow building blocks with internal projections located in the cavity, but in the previously known blocks of this type the internal projections are arranged co-axially with the primary projections.

In contradistinction to prior art, however, the secondary projections **22** according to this invention are arranged co-axially with the centre of a square defined by four primary projections **21**, the cross section of the secondary projections **22** being defined in such a manner that it touches the cross sections of the four primary projections defining said square when said cross-sections are geometrically projected normal to the bottom wall.

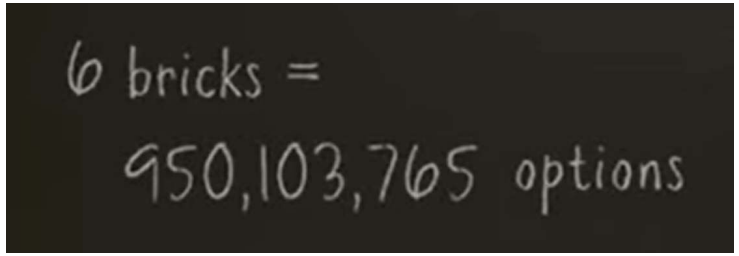
FIG. 8.



US Patent 3,005,282

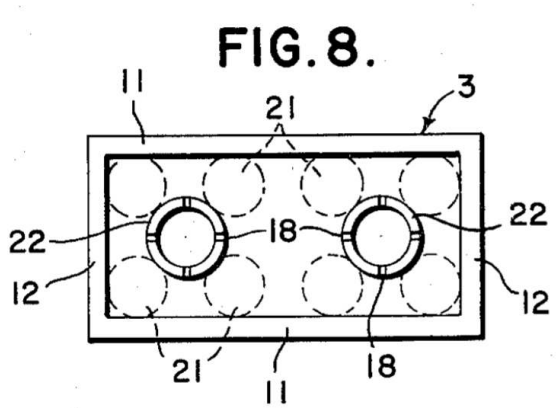
Case Study 1: Creative Play

The „Lego System“ concept introduced **architectural constraints** to permit a specific desired non-functional profile: **Huge Network Effects**

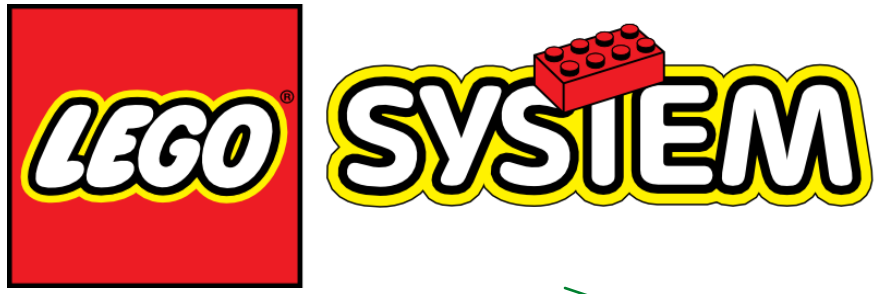


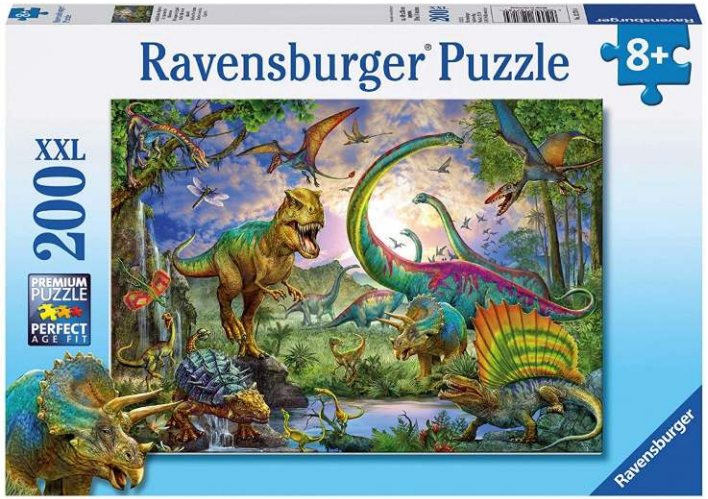
- **Reinforced Interfaces** support **durability** of designs
- **Uniform Interfaces** enable **creative recombinations**

This non-functional profile proved **highly successful on the market**!

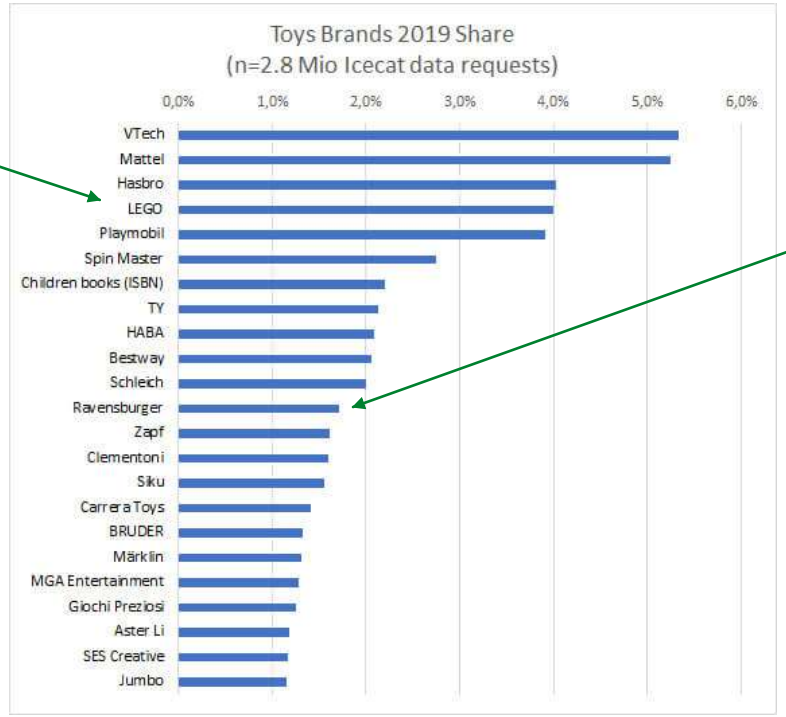


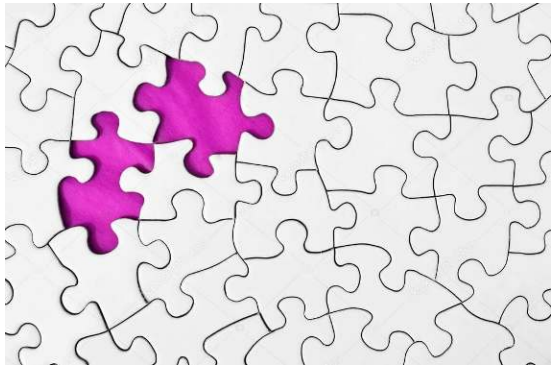
Other non-functional profiles are highly competitive as well!



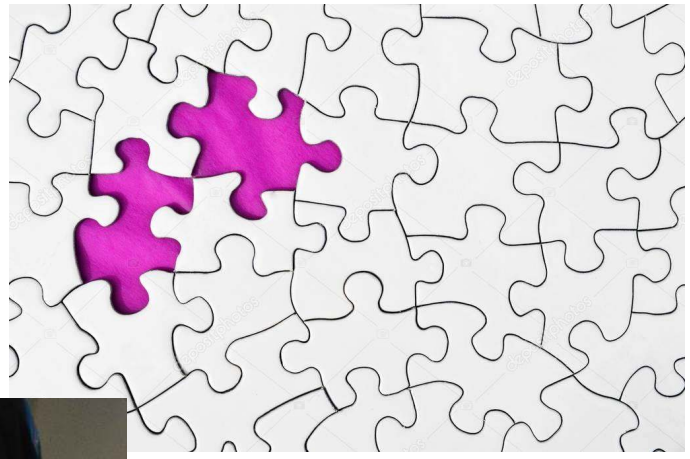








Play and Discuss!

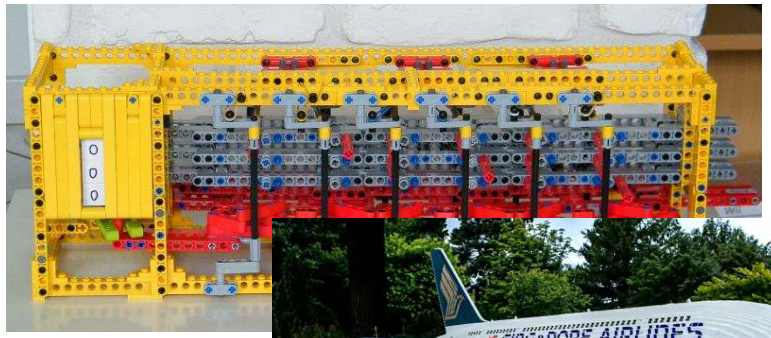




No uniform interfaces? Of course there are!

Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

Case Study 1: Creative Play



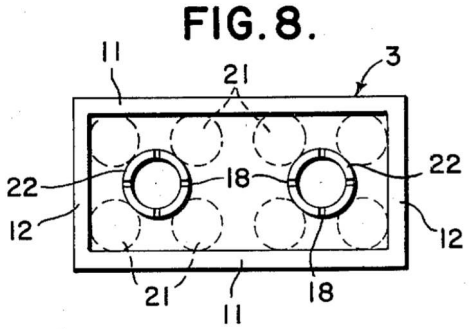
https://www.nico71.fr/lego-computer-dig

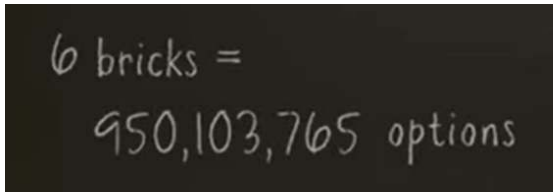


https://aviationhumor.net/the-largest-lego-airplane-in-the-world/

Non-functional **Requirements**

Many creative recombinations and **cheap sturdy blocks**



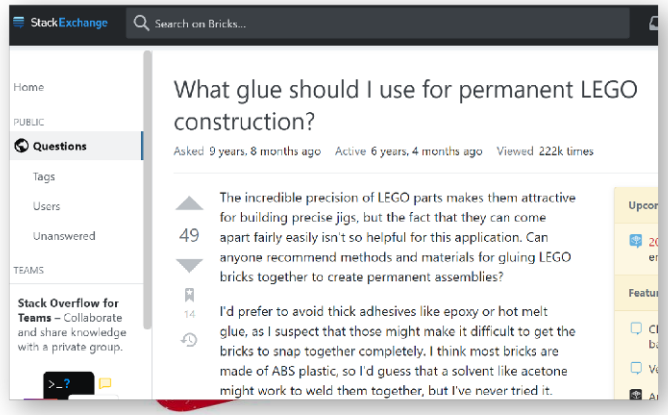


Architectural **Constraints**

Reinforced Interfaces
Uniform Interfaces

Uniform Interface?



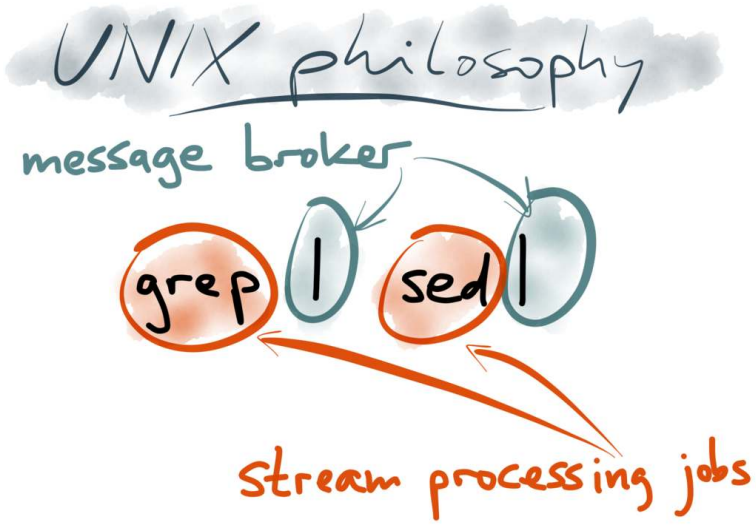


https://bricks.stackexchange.com/questions/1037/what-glue-should-i-use-for-permanent-lego-construction/1038

Architectural **Drawbacks**

Durability of Constructions
Freedom of Elements

Consider a few more examples...



bologna process

LEGO SYSTEM

Data Unit	Layer
data	application
segments	transport
packets	network
frames	data link
bits	physical



Case Study 2: Architecture

Architectural elements (windows, walls, arches) are supplied by **different stakeholders** (masons, carpenters, window builders).

The elements **interact** through structural mechanics and structural dynamics.

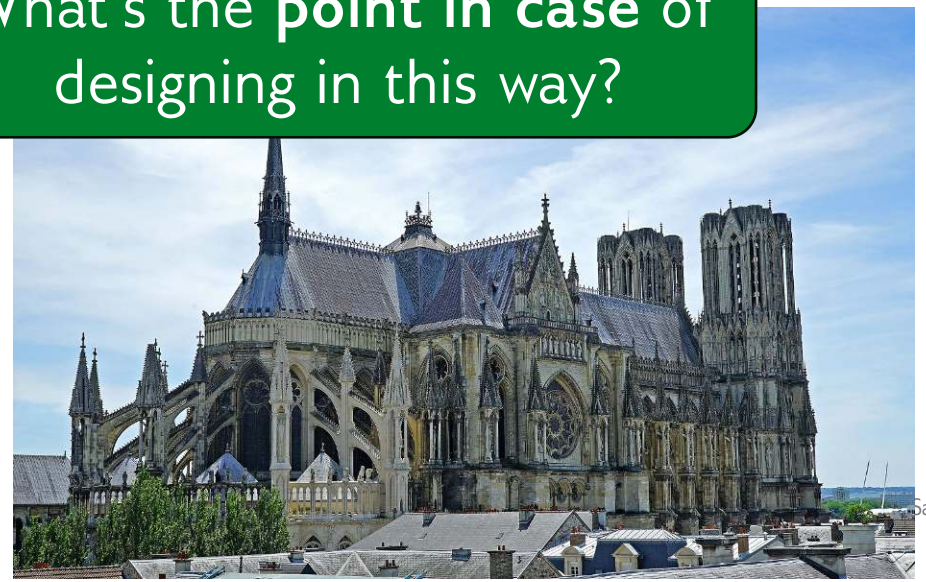
Structural engineers provide **coordinated design constraints** that comply with the functional and non-functional requirements.

These coordinated constraints **might become** a recognized architectural style.

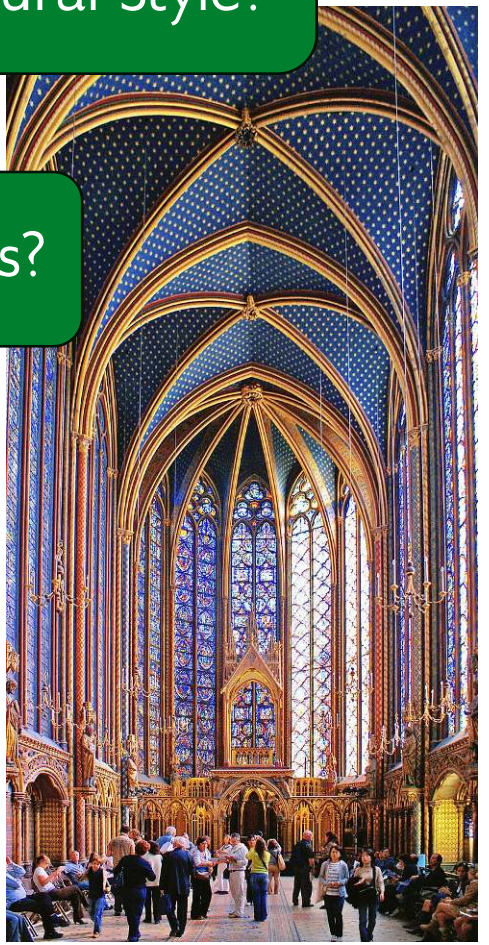
What are the **defining constraints** of the Gothic Architectural Style?

What do you think **why** that is?

What's the **point in case** of designing in this way?



Notre-Dame de Reims, Wikimedia Commons



Sainte-Chapelle, Paris. Wikipedia: Gothic architecture

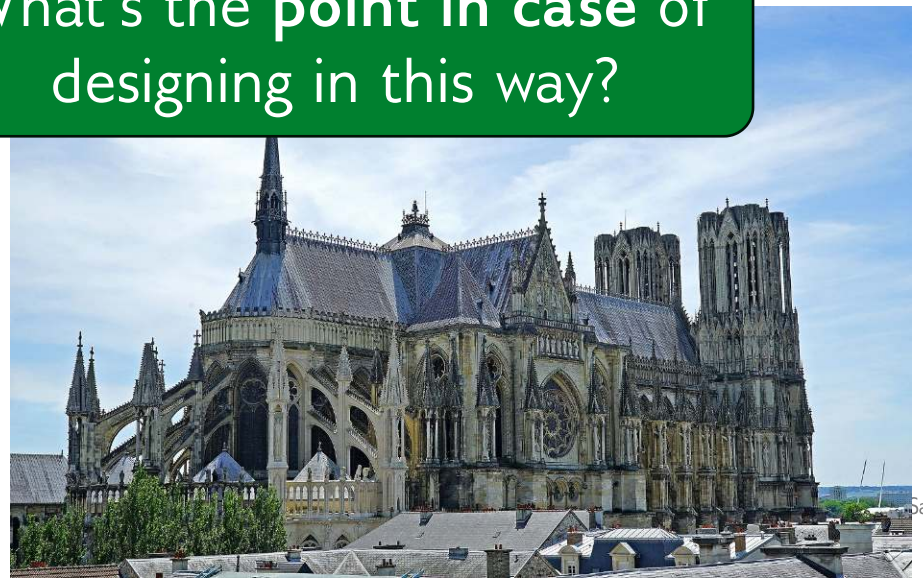
Desired Non-Functional Profile

Non-Functional Profile
Low Cost
Lightweight Construction
Bright Spaces
Tall Structures
Rapid Construction
Aesthetics
…
…
…
…
…
…

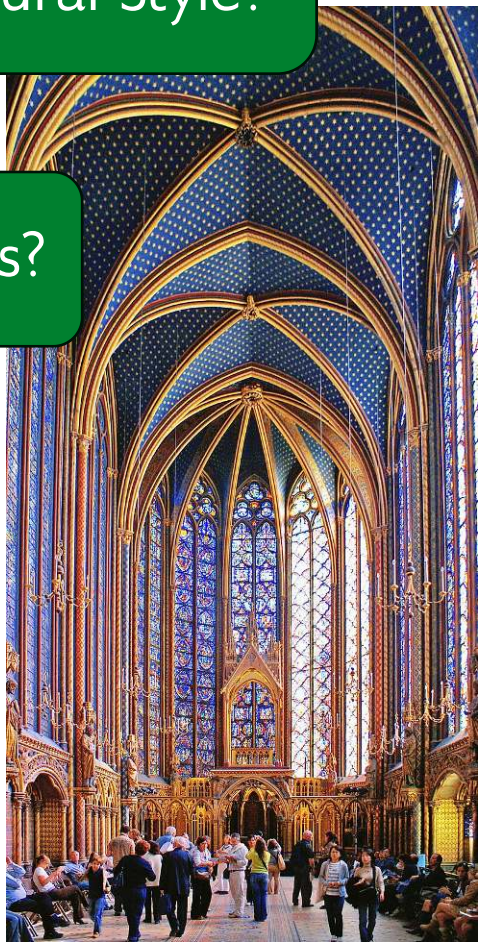
What are the **defining constraints** of the Gothic Architectural Style?

What do you think **why** that is?

What's the **point in case** of designing in this way?



Notre-Dame de Reims, Wikimedia Commons



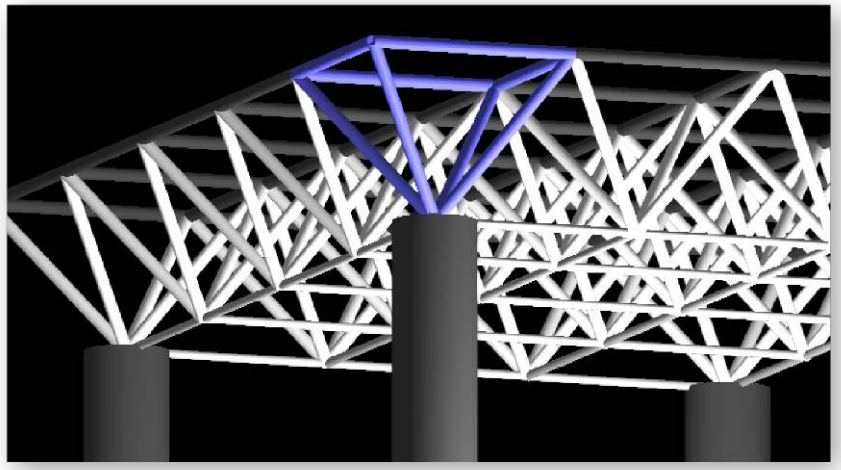
Sainte-Chapelle, Paris. Wikipedia: Gothic architecture

Structural Challenges

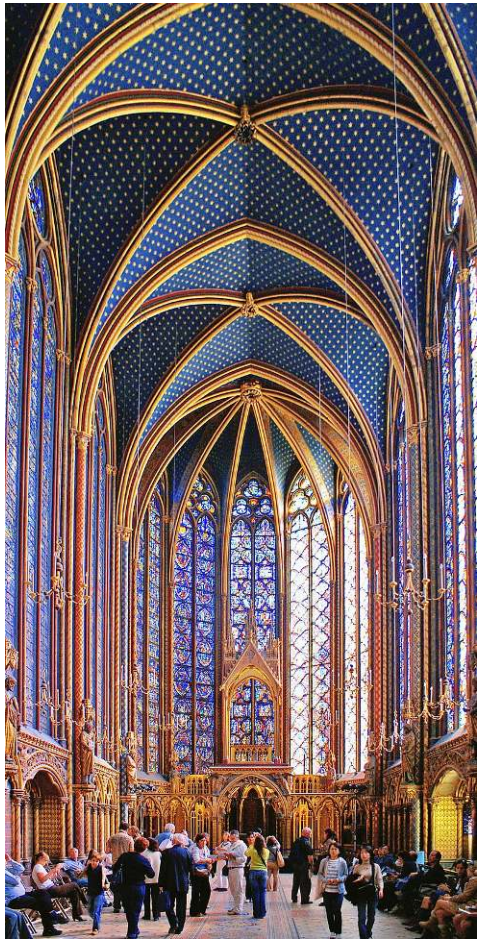
Non-Functional Profile
Low Cost
Lightweight Construction
Bright Spaces
Tall Structures
Rapid Construction
Aesthetics
…
…
…
…
…
…

Problem #1: Fragile materials! Can **windows** provide enough support to the ceiling?

Problem #2: Concentrated strain! Can the foundation provide enough support for the walls?



Space frame, Wikipedia

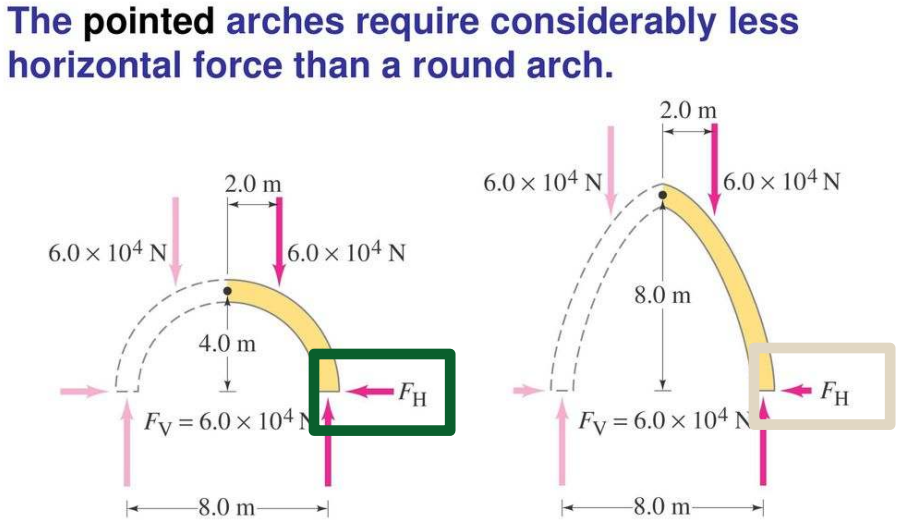


Sainte-Chapelle, Paris. Wikipedia: Gothic architecture

Architectural Constraints

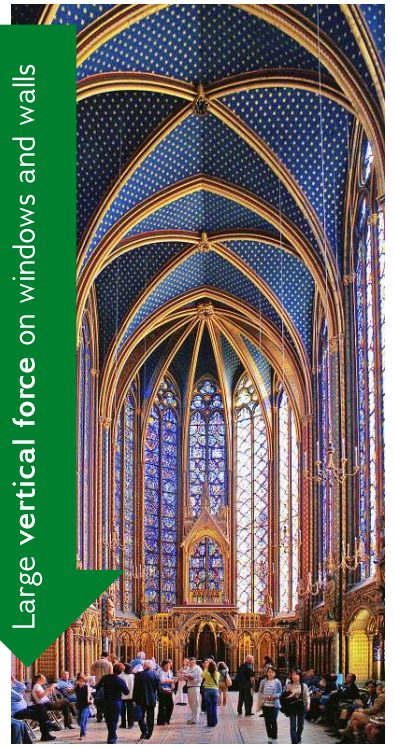
Pointed Arches

- Support spaciousness (allow higher ceilings)
- Support illumination (allow larger windows)
- Take horizontal strain off walls
- Take vertical strain from glass (shifted to walls)



Copyright © 2009 Pearson Education, Inc.

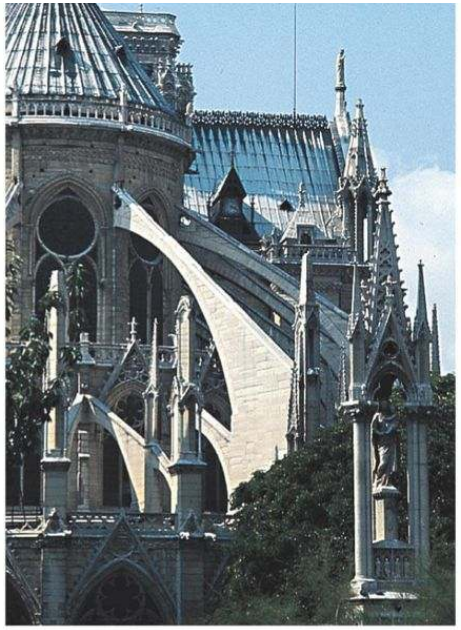
https://slideplayer.com/slide/14163033/



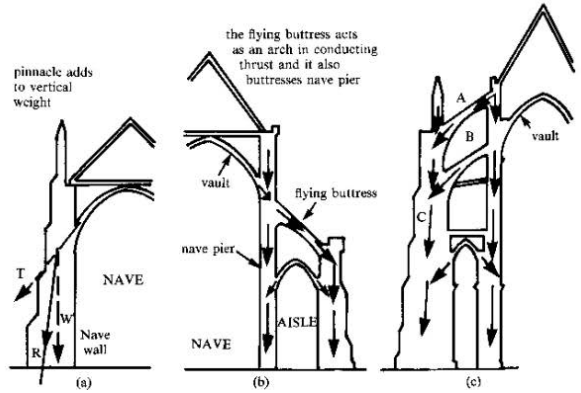
https://en.wikipedia.org/wiki/Gothic_architecture#/media/File:Sainte_Chapelle_-_Upper_level_1.jpg

Flying Buttresses

- Vertical strain is shifted to walls
- Required to spread weight across larger foundation



Copyright © 2009 Pearson Education, Inc.



https://gothic-vaults-edinburgh-thesis.weebly.com/about-the-project.html

Induced Trade-Offs

Non-Functional Profile
Low Cost
Lightweight Construction
Bright Spaces
Tall Structures
Rapid Construction
High Structural Integrity
Aesthetics
...
...
...
...
...

Bruce Feldman
Leasing and Managing Agent for NYC landlords. Real Estate Broker in New York State. · Author ... · 12y

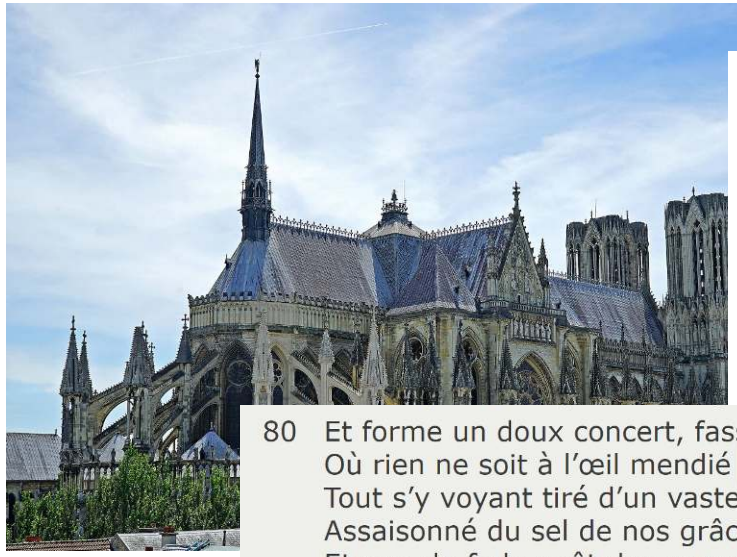
Very generally I'd say it would cost at least 2-3 times the cost of more modern, conventional designs. And if one wanted to design and construct an authentic replica of gothic collegiate buildings, with concrete walls several feet thick and traditional gothic elements, the cost might be double that figure or more.

Upvote · 2 1

Naggarothi • vor 4 Jahren •

Not much more than a normal house especially if you have less windows

1 Antworten Auszeichnen Teilen ...



Byzantine Ambassador
@byzantinepower

Fine, I'll say it. Flying buttresses are an ugly architectural solution.

12:52 PM · Aug 16, 2019 · Twitter for Android

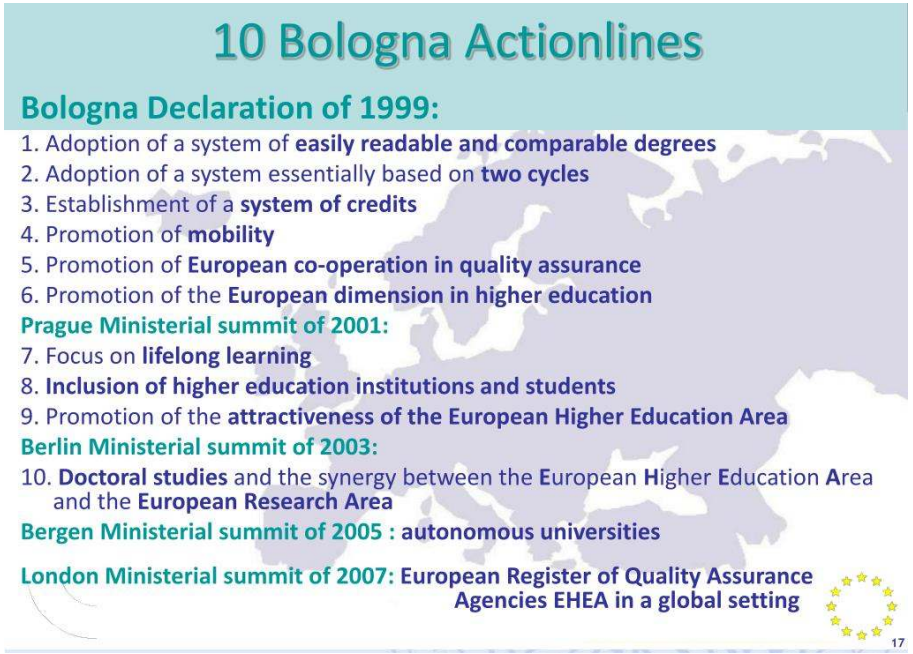
2 Retweets **1** Quote Tweet **19** Likes

Reply

80 Et forme un doux concert, fasse un beau tout-ensemble [i] ,
Où rien ne soit à l'œil mendié [18] , ni redit ;
Tout s'y voyant tiré d'un vaste fonds d'esprit,
Assaisonné du sel de nos grâces antiques,
Et non du fade goût des ornements gothiques [19] :
85 Ces monstres odieux des siècles ignorants,
Que de la barbarie ont produits les torrents ;
Quand leur cours inondant presque toute la terre,
Fit à la politesse une mortelle guerre,
Et de la grande Rome abattant les remparts,

Molière, La Gloire (1669)

Case Study 3: The Bologna Process







Non-Functional Profile
Interoperability
Responsiveness
Scalability
Extensibility
Autonomy
Testability
Consistency
Adaptability
Mobility
…
…
…

Induced Architectural Constraints

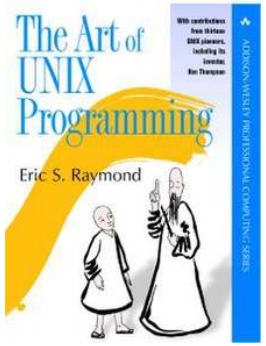
- Standardized Credits («ECTS»)
- Standardized Cycles and Degrees («Bachelor/Master»)
- Mandatory Acceptance of Degrees from other Institutions

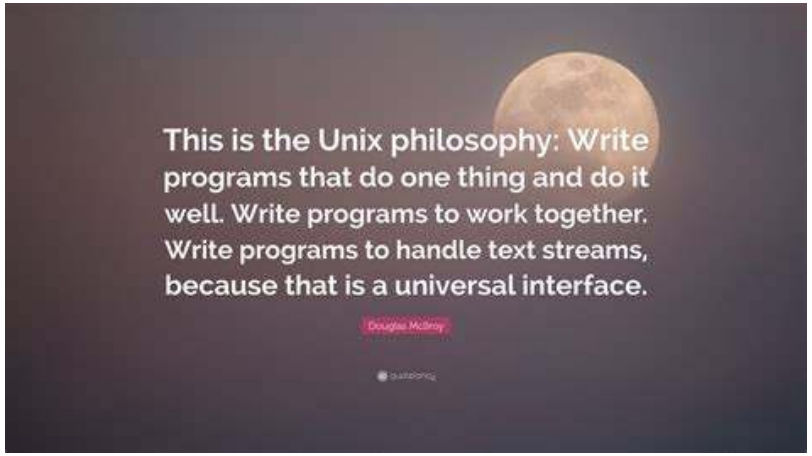
And what are the **trade-offs**?

Heavily **constrained autonomy** of individual university programs. **Low responsiveness** to educational developments.

Case Study 4: The Unix Philosophy

- Build **modular programs**
- Write readable programs
- Use **composition**
- Separate mechanisms from policy
- Write **simple programs**
- Write **small programs**
- Write transparent programs
- Write robust programs
- Make data complicated when required, not the program
- Build on potential users' expected knowledge
- Avoid unnecessary output
- Write programs which fail in a way that is easy to diagnose
- Value developer time over machine time
- Write abstract programs that generate code instead of writing code by hand
- Prototype software before polishing it
- Write **flexible and open programs**
- Make the program and protocols **extensible**





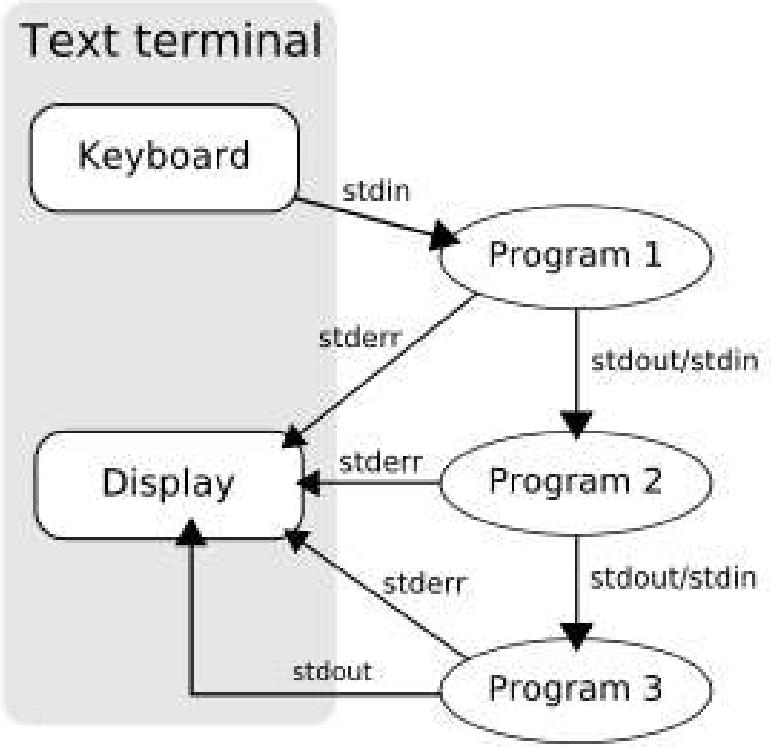
Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability

Architecturally: **Contribution to Toolbox Philosophy of Unix**

Case Study 4: The Unix Philosophy

```
SMayer@NB-21280 MINGW64 .../Teaching/ASSE/FS22/ (master)
$ ls -l | grep key | less
```

```
process1 | process2 | process3
```



Induced Architectural Constraints

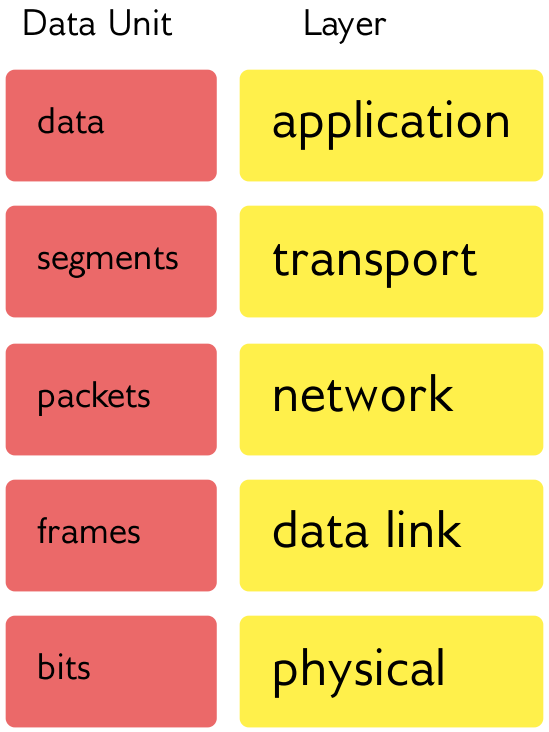
- Read input from **standard input**
- Write (main) output to **standard output**

And what are the **trade-offs**?

Low design freedom of individual elements.
Optimization across processes is impossible without sacrificing the uniform interface

Pointer Lecture 11: What's different when we instead consider systems ... ↗

Case Study 5: Computer Networking



Individual layers should be exchangeable / reusable / recombinable

Applications should **not depend on physical carrier medium**

Core intent was to **circumvent AT&T's monopoly** on telephone cables

And what are the **trade-offs**?

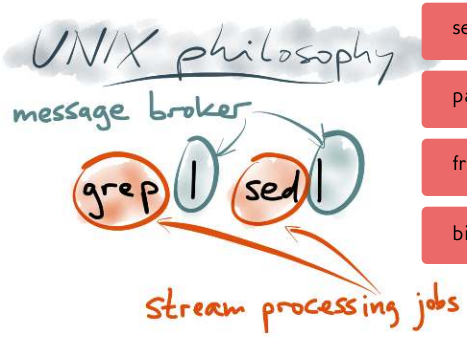
Induced Architectural Constraint

- **Layered** networking stack
- Strict **separation of concerns**

Optimization across layers is impossible without sacrificing separation of concerns. Points in case: QUIC, CoAP, BLE !

Non-Functional Profile
Reliability
Responsiveness
Availability
Fault Tolerance
Scalability
Elasticity
Data integrity
Data consistency
Adaptability
Extensibility
Testability
Deployability



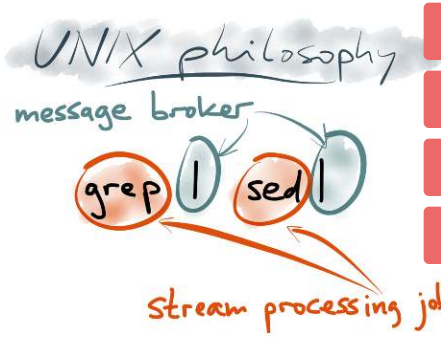


Data Unit	Layer
data	application
segments	transport
packets	network
frames	data link
bits	physical



Case	Constraints	Desired NFP	Trade-Offs
Lego	Uniform Interfaces (patented)	Interoperability, Extensibility	Design Freedom, Durability
Gothic Style	Pointed Arches, Flying Buttresses, Rib Vaults	Brightness	Cost, Aesthetics
Bologna	Uniform Interfaces (ECTS, BSc/MSc)	Interoperability, Scalability	Autonomy
Unix Philosophy	Uniform Interfaces (pipe-and-filter)	Interoperability, Extensibility	Design Freedom, Efficiency
Computer Networking	Layers, Uniform Interfaces	Extensibility, Adaptability	Performance

No coincidence!



Data Unit	Layer
data	application
segments	transport
packets	network
frames	data link
bits	physical



Our examples illustrate that specific **architectural constraints** are placed **consciously** to achieve **specific non-functional profiles**.

When you design a computer system, we want you to do the same: **consciously select fitting architectural constraints to achieve the non-functional profile you desire**

Case	Constraints	Desired NFP	Trade-Offs
Lego	Uniform Interfaces (patented)	Interoperability, Extensibility	Design Freedom, Durability
Gothic Style	Pointed Arches, Flying Buttresses, Rib Vaults	Brightness	Cost, Aesthetics
Bologna	Uniform Interfaces (ECTS, BSc/MSc)	Interoperability, Scalability	Autonomy
Unix Philosophy	Uniform Interfaces (pipe-and-filter)	Interoperability, Extensibility	Design Freedom, Efficiency
Computer Networking	Layers, Uniform Interfaces	Extensibility, Adaptability	Performance

No coincidence!

Let‘s look at one of the mo**st interoperable technical system** we know today...

...which is, at the same time, the **most prominent information system** globally.

Select Seven Characteristics!

A **globally connected** research association is, each year, welcoming **thousands of researchers** who bring their **diverse data and results** in **diverse formats** and on **diverse computer systems**. The researchers need to be enabled to **integrate** these results and we **don't know** which formats will come next.

- Users: Tens of thousands of researchers world-wide
- Requirements: See above!
- Context:
  - Diverse computer systems with heterogeneous data formats
  - Open-ended fundamental research projects

Select Seven!

☐ deployability ☐ adaptability ☐ scalability
☐ learnability ☐ fault tolerance ☐ recoverability
☐ availability ☐ workflow ☐ reliability
☐ performance ☐ maintainability ☐ elasticity
☐ extensibility ☐ responsiveness ☐ interoperability
☐ data integrity ☐ testability ☐ cost

Select Seven Characteristics!

A **globally connected** research association is, each year, welcoming **thousands of researchers** who bring their **diverse data and results** in **diverse formats** and on **diverse computer systems**. The researchers need to be enabled to **integrate** these results and we **don't know** which formats will come next.

- Users: Tens of thousands of researchers world-wide
- Requirements: See above!
- Context:
  - Diverse computer systems with heterogeneous data formats
  - Open-ended fundamental research projects

My Selection

☐ **deployability** ☐ **adaptability** ☐ **scalability**
☐ **learnability** ☐ **fault tolerance** ☐ recoverability
☐ availability ☐ workflow ☐ reliability
☐ performance ☐ maintainability ☐ elasticity
☐ **extensibility** ☐ responsiveness ☐ **interoperability**
☐ data integrity ☐ testability ☐ cost

What is the Web?

Scalability

An Internet-scale Distributed Hypermedia System!

Fault Tolerance

"constantly running, always changing, anarchically accessed, and independently deployed"

Adaptability & Extensibility

Interoperability & Deployability

"readily usable by anyone"

Learnability

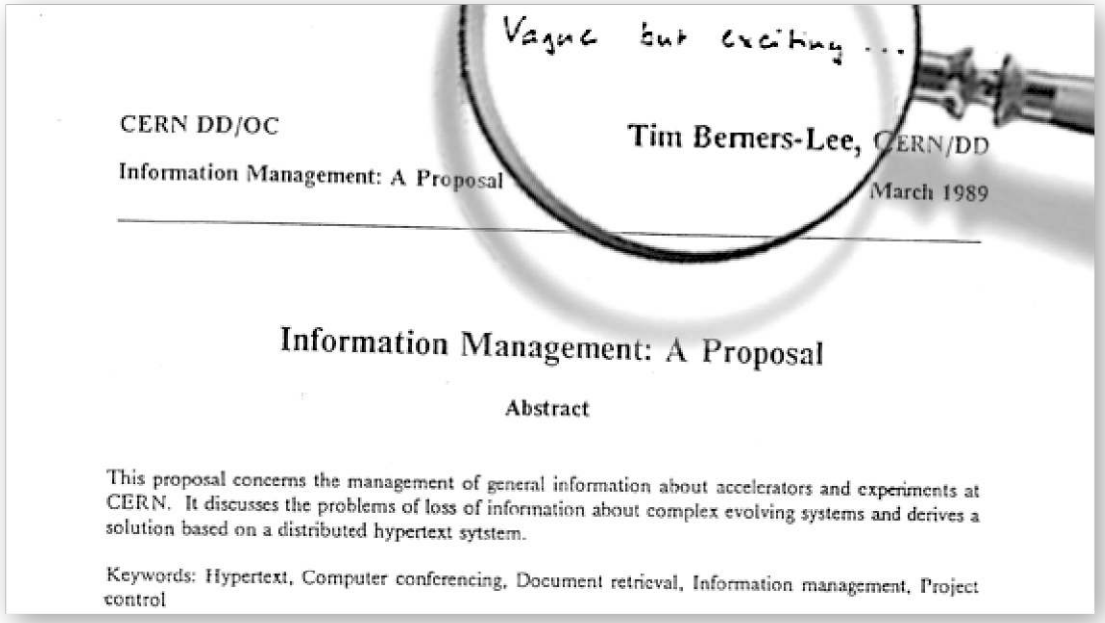
- R. Fielding

Roy T. Fielding, Rohit Khare, Richard N. Taylor. FSE'17 Keynote

The World Wide Web's NFPs

At CERN, Tim Berners-Lee's intention is to create an **information management system** that would improve the management of scientific data about CERN's accelerators and experiments, especially given CERN's **complex evolving systems**.

He is aware of the work of the **Hypertext** community that is researching the management of non-linear information and submits a **proposal for an information management system** at CERN to his boss, M. Sendall.


Vague but exciting ...

CERN DD/OC

Tim Berners-Lee, CERN/DD

Information Management: A Proposal

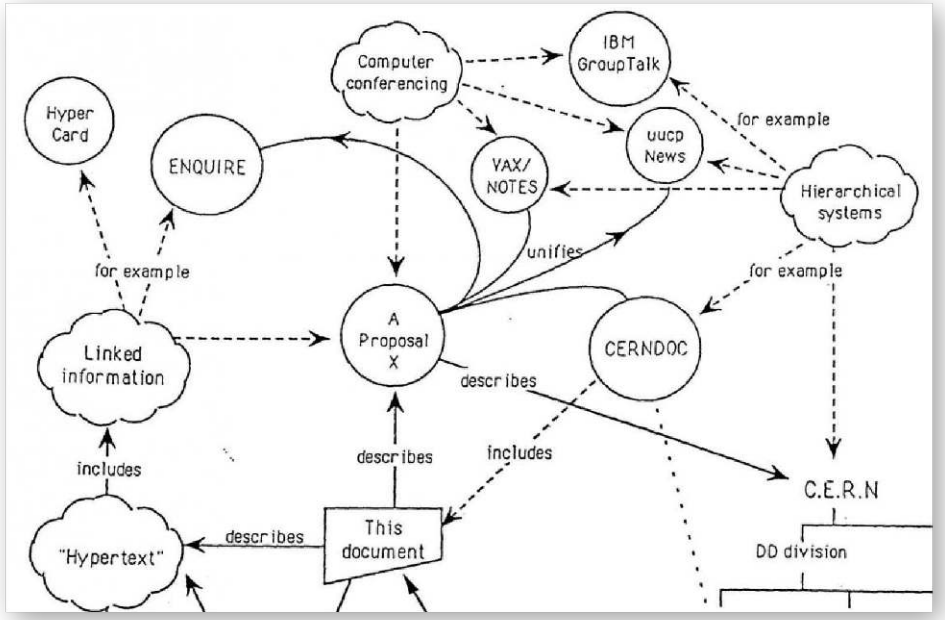
March 1989

Information Management: A Proposal

Abstract

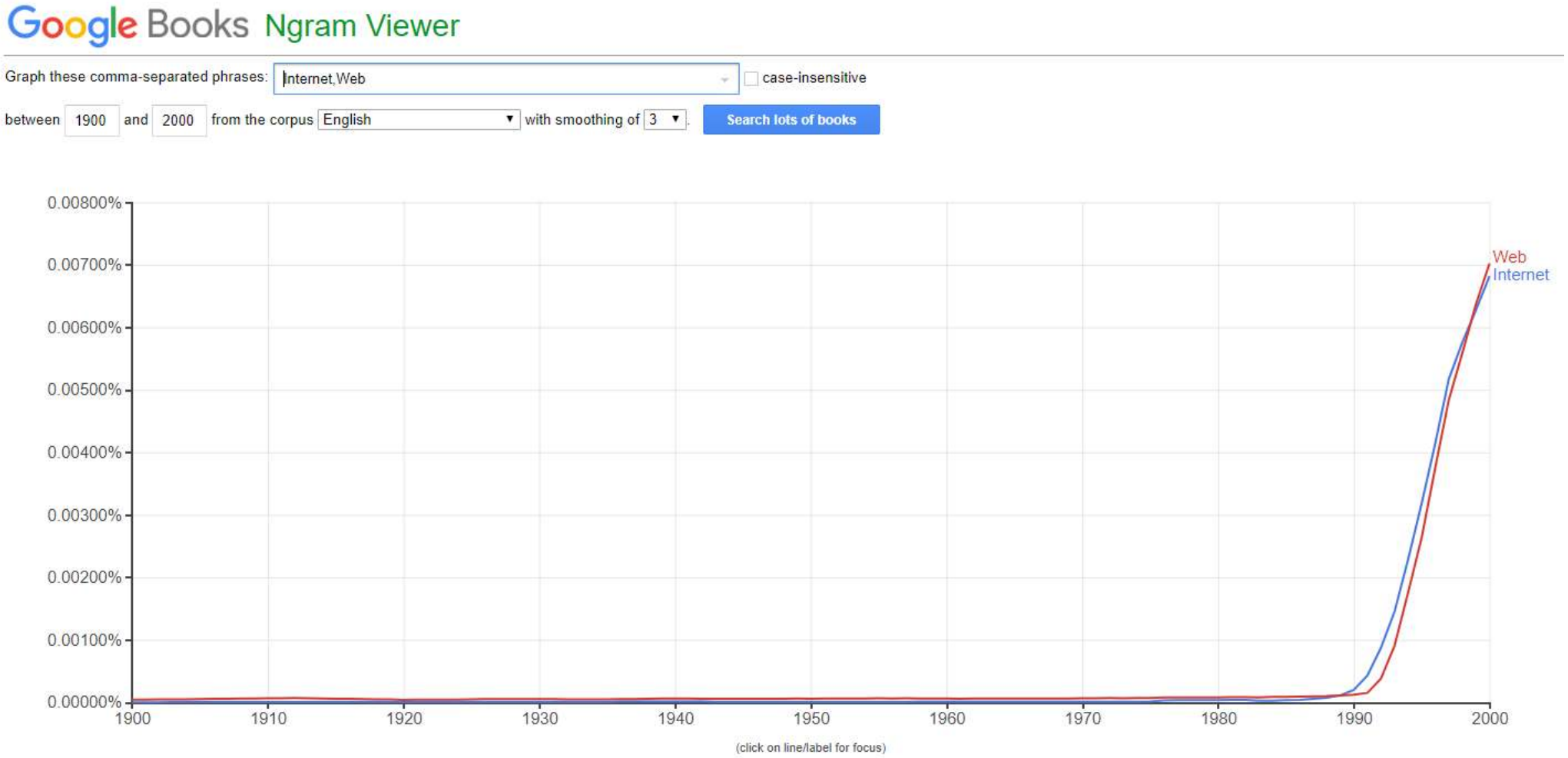
This proposal concerns the management of general information about accelerators and experiments at CERN. It discusses the problems of loss of information about complex evolving systems and derives a solution based on a distributed hypertext sytstem.

Keywords: Hypertext, Computer conferencing, Document retrieval, Information management, Project control



http://info.cern.ch/Proposal.html

A World Wide Wild Success



Coincidence?

Was the Web **the only contender** for a hypermedia-based information system?

Was it easy for „the Web“ to make its case and **dominate** distributed information systems?

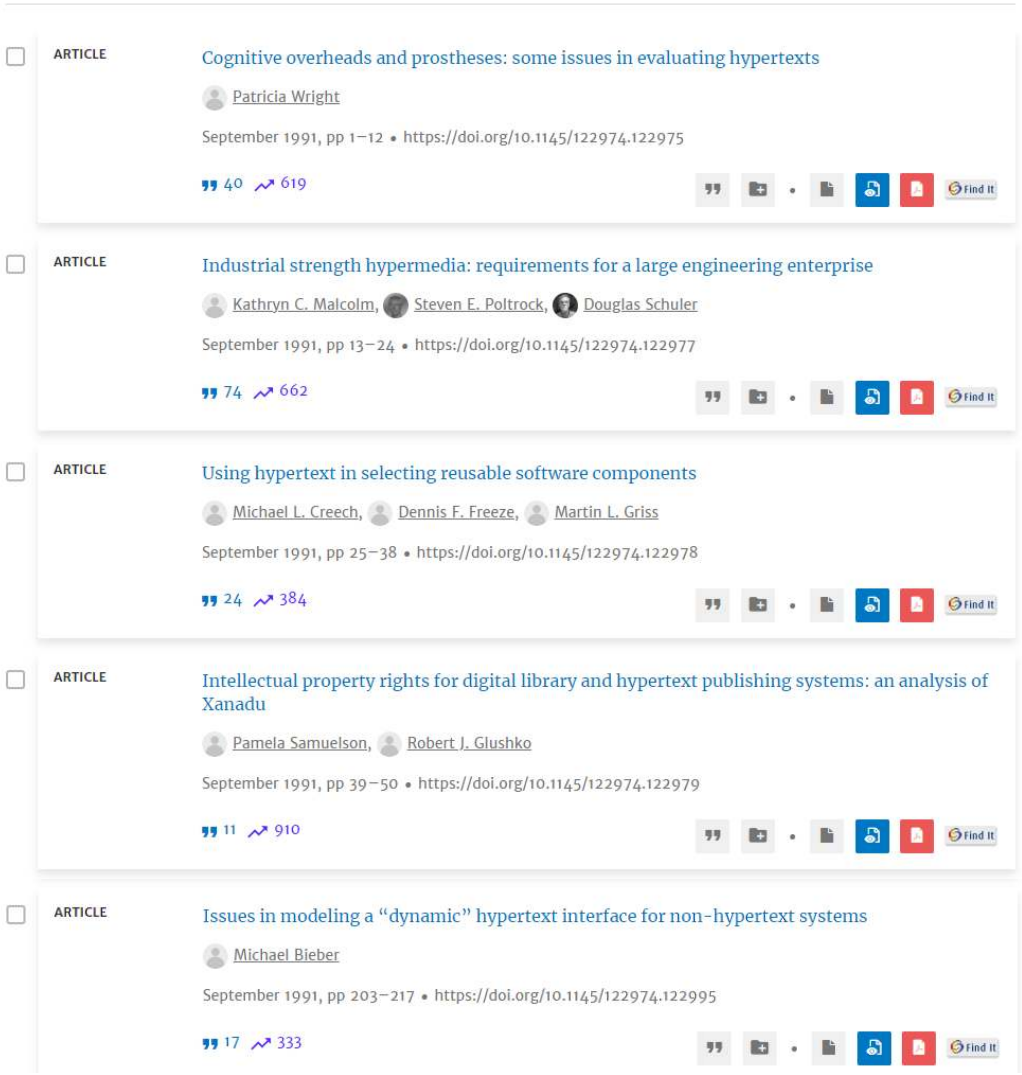
The World Wide Web's NFPs

Berners-Lee defines the terms **URI** (then UDI), **HTTP**, and **HTML**, and develops the first components and tools for the Web (browser, editor, server, line-mode browser) in 1990.

Tim Berners-Lee submits a paper on the World Wide Web to the *Hypertext '91 Conference*.

> "Individual links are allowed to break **so the entire Web does not.**" – T. Berners-Lee

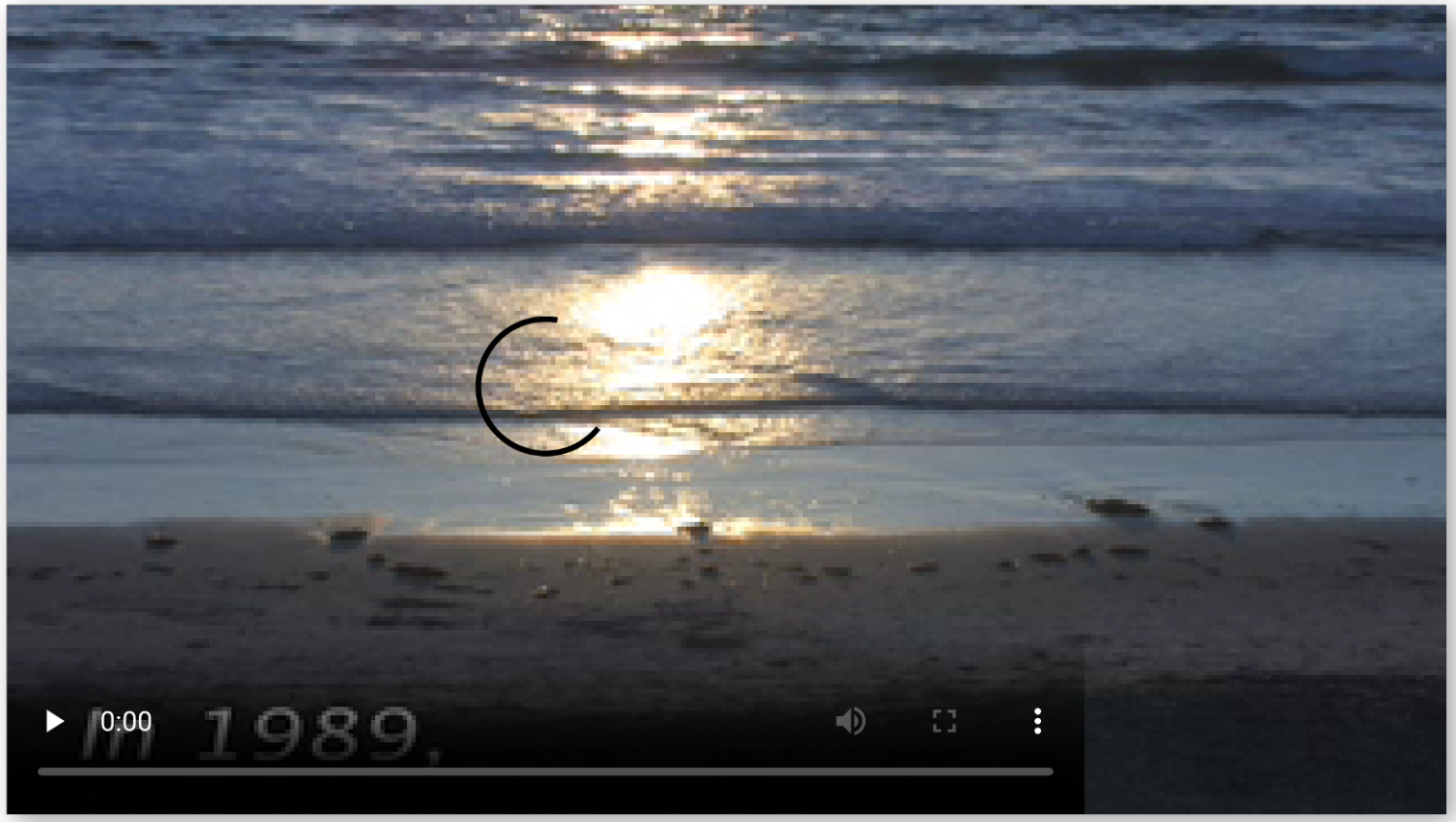
The paper is **rejected as an oral presentation** and only accepted as a **poster** contribution.



W3C: How It All Started · Hypertext '91 proceedings

It‘s again about Trade-offs!

Emphasis on **Recoverability, Availability, Reliability, Integrity**
Good reasons to object to “individual links are allowed to break”!

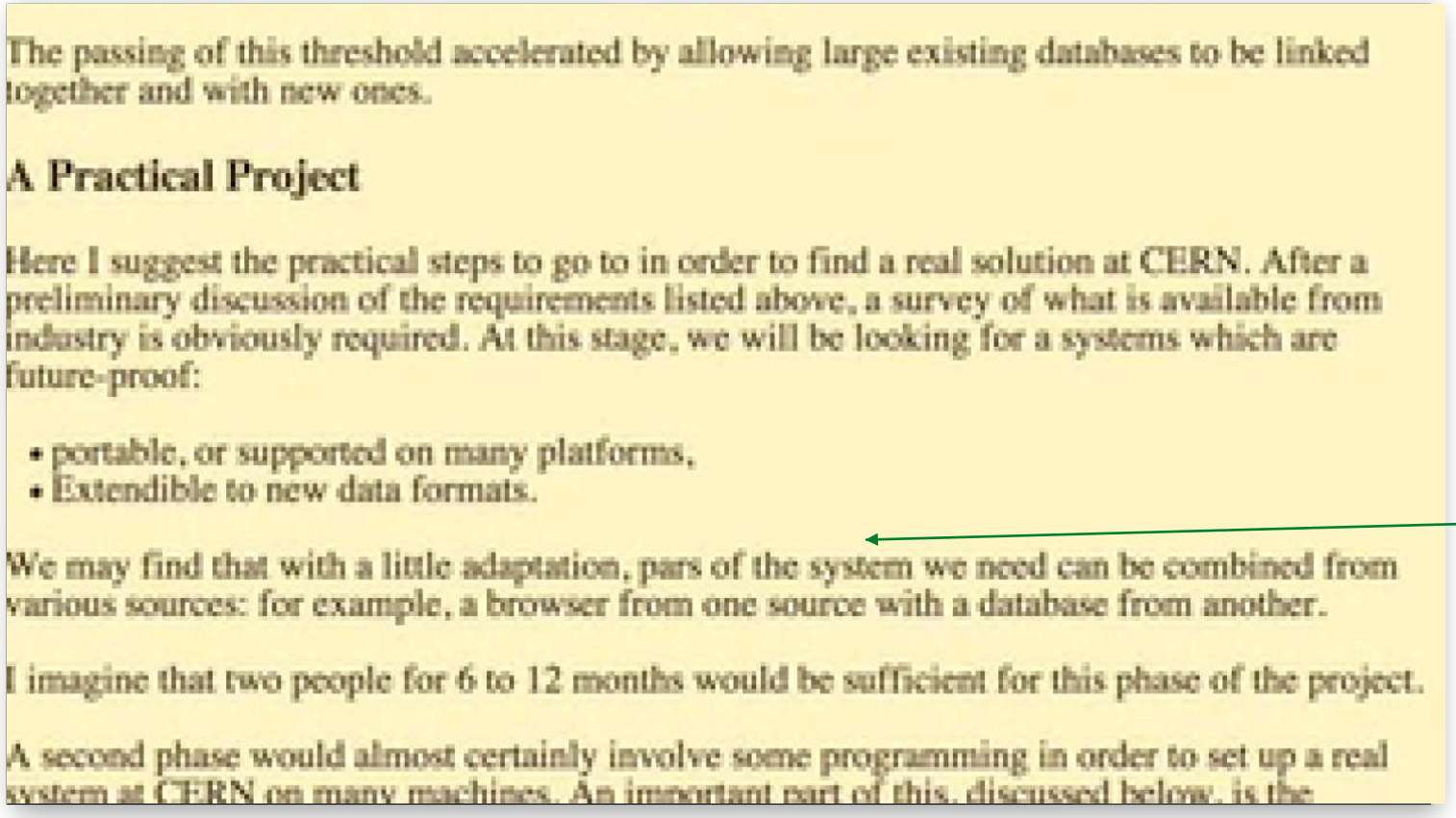


Very different emphasis and non-functional profile!

Mark Frisse, MD, MS, is Professor Emeritus of Biomedical Informatics in the School of Medicine at Vanderbilt University. His work focuses on the intersection between health care informatics, economics, policy, and health care transformation. His primary research is directed toward an understanding of economic sustainability and toward the development of technical and administrative measures to enable effective care coordination and to ensure the integrity of security and privacy efforts.

It‘s again about Trade-offs!

Emphasis on **Recoverability, Availability, Reliability, Integrity**
Good reasons to object to “individual links are allowed to break”!



The passing of this threshold accelerated by allowing large existing databases to be linked together and with new ones.

A Practical Project

Here I suggest the practical steps to go to in order to find a real solution at CERN. After a preliminary discussion of the requirements listed above, a survey of what is available from industry is obviously required. At this stage, we will be looking for a systems which are future-proof:

- portable, or supported on many platforms,
- Extendible to new data formats.

We may find that with a little adaptation, pars of the system we need can be combined from various sources: for example, a browser from one source with a database from another.

I imagine that two people for 6 to 12 months would be sufficient for this phase of the project.

A second phase would almost certainly involve some programming in order to set up a real system at CERN on many machines. An important part of this, discussed below, is the

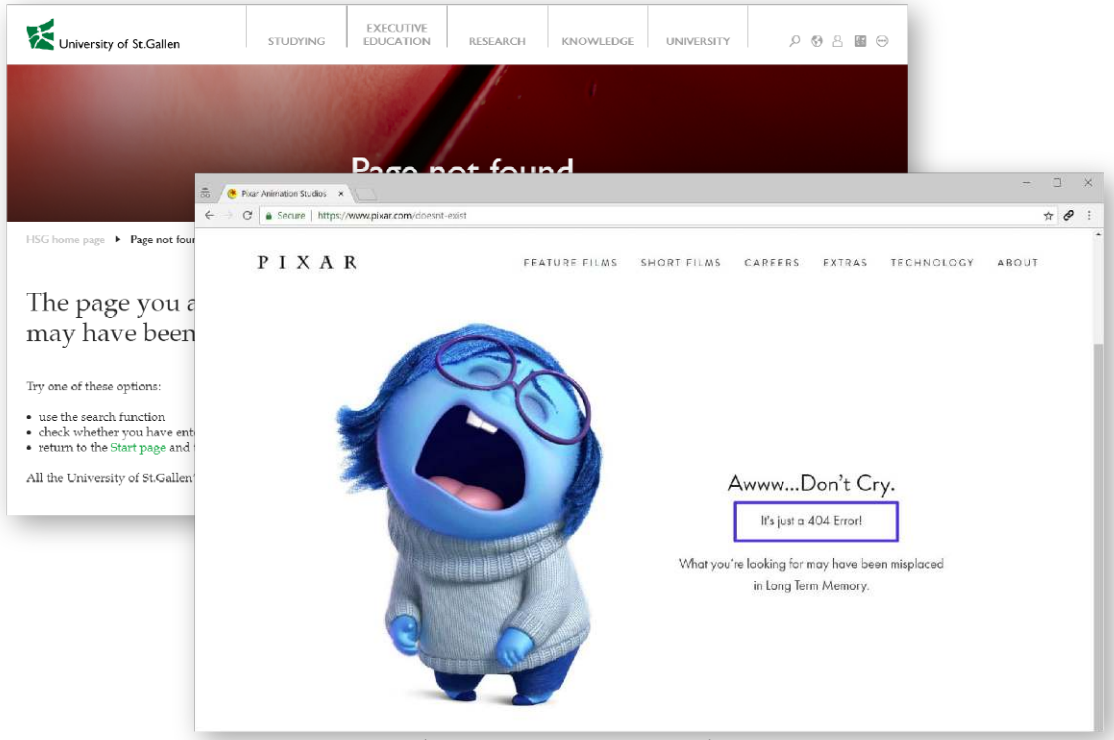
Very different emphasis and non-functional profile!

Mark Frisse, MD, MS, is Professor Emeritus of Biomedical Informatics in the School of Medicine at Vanderbilt University. His work focuses on the intersection between health care informatics, economics, policy, and health care transformation. His primary research is directed toward an understanding of economic sustainability and toward the development of technical and administrative measures to enable effective care coordination and to ensure the integrity of security and privacy efforts.

It‘s again about Trade-offs!

Emphasis: Adaptability, Scalability, Extensibility, Interoperability

"Individual links are allowed to break **so the entire Web does not.**" – T. Berners-Lee



Emphasis: Recoverability, Availability, Reliability, Integrity

"I was part of the camp who really believed that hypertext systems had to be **closed** systems with **bi-directional links**."

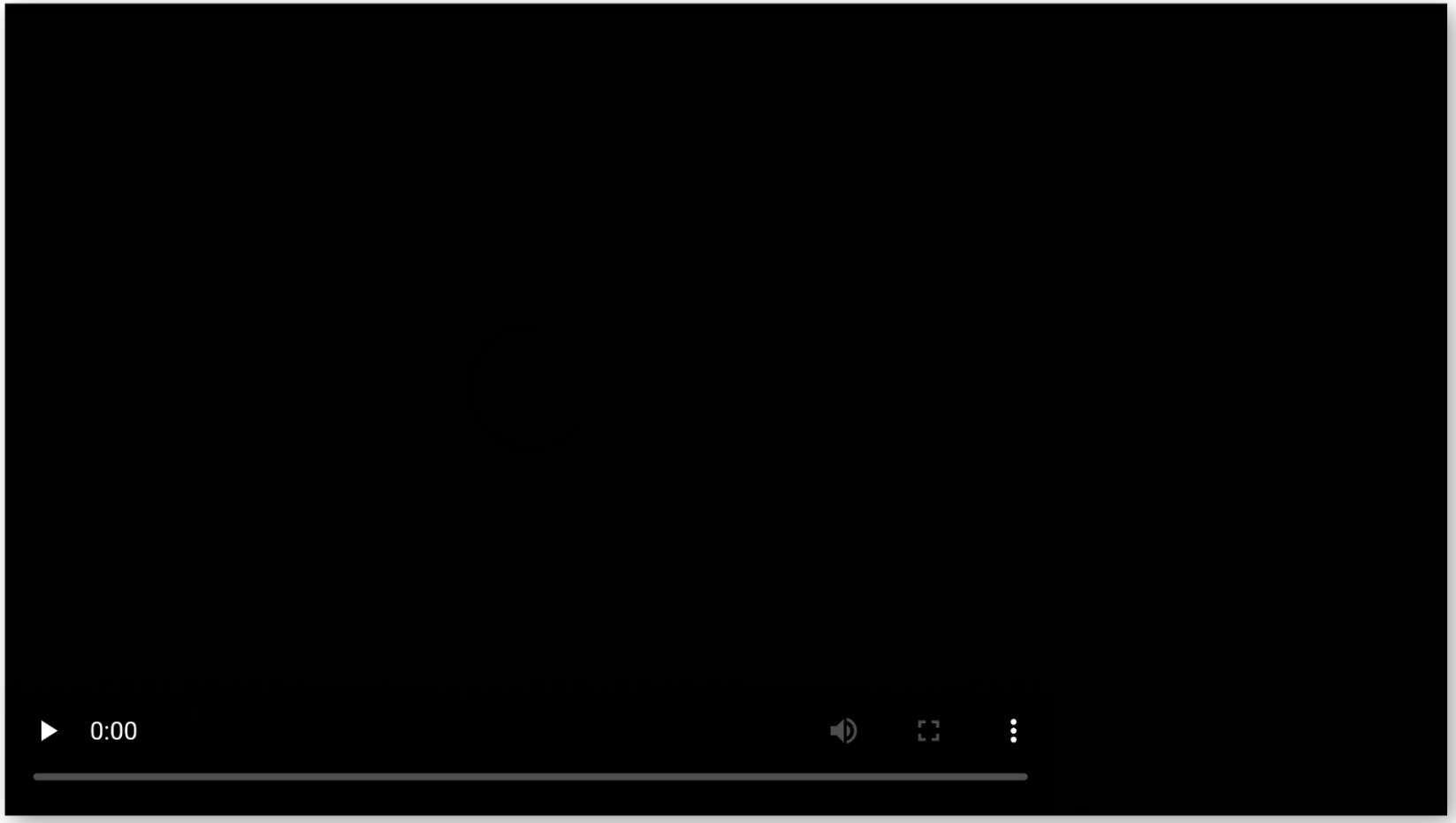
M. Frisse

https://archive.org/embed/Munnecke-MarkFrissesMessageToTimBernersLee313

"Today's popular software simulates paper. The World Wide Web (another imitation of paper) **trivialises our original hypertext model with one-way ever-breaking links and no management of version or contents.**"

Project Xanadu

It‘s again about Trade-offs!



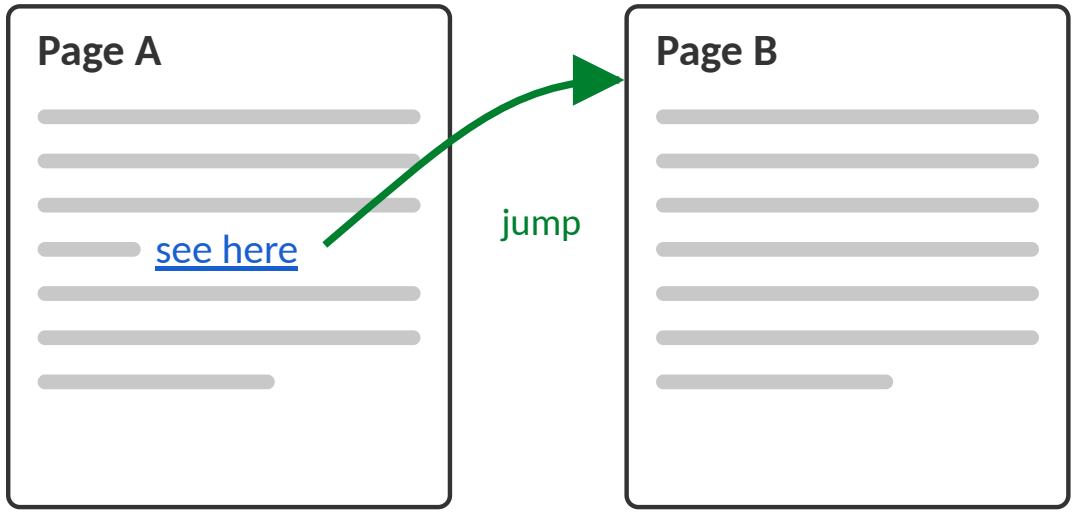
sole. And I am trying to work out the forms of writing of the future. Hypertexts.

Hypertexts: new forms of writing, appearing on computer screens, that will branch or perform at the reader's command. A hypertext is a non-sequential piece of writing; only the computer display makes it practical. Somewhere between a book, a TV

Theodor Holm Nelson (born June 17, 1937) is an American pioneer of information technology, philosopher, and sociologist. He coined the terms *hypertext* and *hypermedia* in 1963[1] and published them in 1965.[2] According to a 1997 *Forbes* profile, Nelson "sees himself as a literary romantic, like a Cyrano de Bergerac, or 'the Orson Welles of software'."[3]

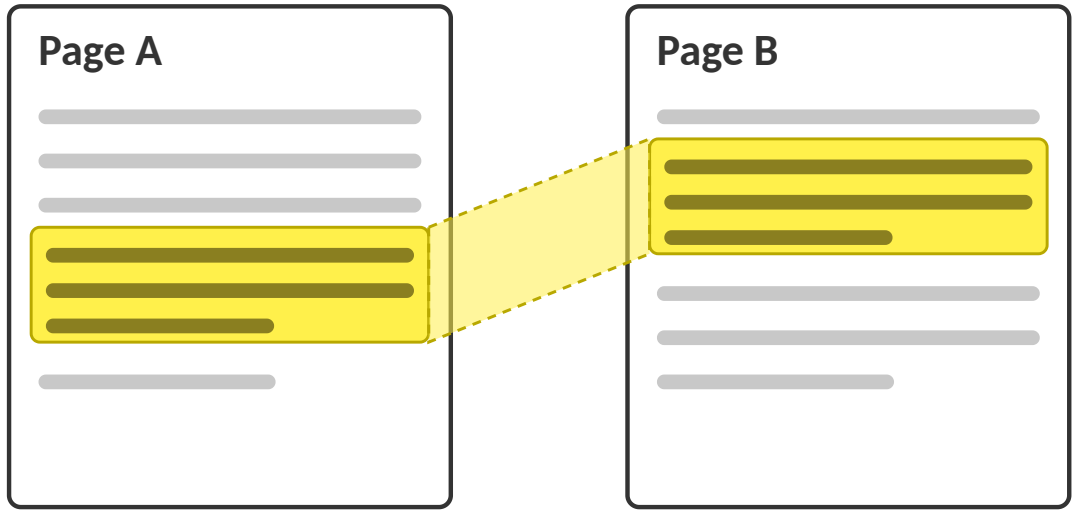
Jump Links (Web) and Transclusion (Xanadu)

Jump link: the Web's hyperlink



- Stored in A, **points one way** to B
- B does not know who links to it; the link can **break**
- To quote B, A **copies** the text

Transclusion: Xanadu's link



- A shows a part of B **by reference**, not by copy
- The connection is **visible from both sides**
- **Author and version** of B stay with the quote

Ted Nelson: Xanalogical Structure, Needed Now More than Ever. ACM Computing Surveys 31(4), 1999

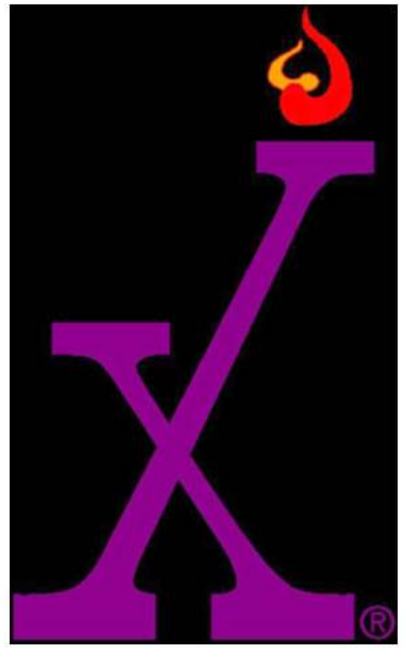
Pointer Lecture 9: Hypertext: Ted Nelson, 1965 ↗

Project Xanadu

PROJECT XANADU®

Founded 1960 * The Original Hypertext Project

"Xanadu®" and the Eternal-Flaming-X logo are registered trademarks of Project Xanadu.



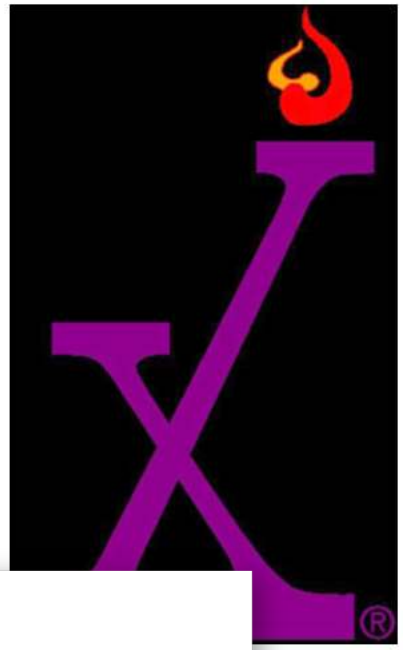
xanadu.com

Project Xanadu

Conventional electronic documents were designed in the 1970s by well-funded conventional thinkers at Xerox PARC, who asked, "How can we imitate paper?" The result is today's electronic document-- Microsoft Word format and the printout format PDF. They imitate paper and emphasize appearance and fonts.

But much earlier, in 1960, the Xanadu project started with a completely different idea: since interactive screens are coming (who else knew?), we asked a different question: How can we IMPROVE on paper?

We foresaw a new screen literature of parallel, interconnected documents.



IMPOSSIBLE ON THE WEB

The World Wide Web took part of our concept, the "hyperlink" (we called it the jump-link, since you can't see where you're going), and left out the visible interconnections (which would allow you to see where you'e going). In his 1989 proposal for the World Wide Web, Tim Berners-Lee said:

> *There are few products which take Ted Nelson's idea of a wide "docuverse" literally by allowing links between nodes in different databases. In order to do this, some standardisation would be necessary.**

* http://www.w3.org/History/1989/proposal.html

The standardization he allowed in his document format only allowed jump-links, with no visible bridges between them.

xanadu.com

Transclusion

To Project Xanadu, that means enacting two types of connection: profuse and unbreakable *deep links* to embody the arbitrary connections that may be made by many authors throughout the world (content links); and *a system of visible, principled re-use*, showing the origins and context of quotations, excerpts and anthologized materials, and content transiting betwen versions (transclusions).

This may be simplified to: connections between things which are *different*, and connections between things which are *the same*. They must be implemented differently and orthogonally, in order that linked materials may be transcluded and vice versa. This double structure of abstracted literary connection -- *content links* and *transclusion* -- constitute xanalogical structure. They are literary connections because they deal with literary boundaries [Nelson 1995]. In the Xanadu designs we have always striven to represent both forms of connection.

Xanalogical structure, needed now more than ever: parallel documents, deep links to content, deep versioning, and deep re-use: ACM Computing Surveys 31(4), 1999

ARTICLE | FREE ACCESS

Xanalogical structure, needed now more than ever: parallel documents, deep links to content, deep versioning, and deep re-use

Author: Theodor Holm Nelson | Authors Info & Claims

ACM Computing Surveys (CSUR), Volume 31, Issue 4es • Pages 33 - es • https://doi.org/10.1145/345966.346033

Published: 01 December 1999 Publication History | Check for updates

58 2,578 | PDF | eReader

Transclusion is what quotation, copying and cross-referencing merely attempt: they are ways that people have had to *imitate* transclusion, which is the true abstract relationship that paper cannot show. Transclusions are not copies and they are not instances, but *the same thing knowably and visibly in more than once place*. This is a simple point which is remarkably difficult to get across. While copies and cross-reference are workarounds in place of transclusion, aliases and caches are *forms* of transclusion.

Not distinguishing between links and transclusions is causing misery everywhere, for instance in lawsuits against having one's page brought into someone else's frame (a form of transclusion) which the lawyers refer to as "linking" -- hopelessly confounding a key difference.

Note also that the famous "trails" of Vannevar Bush's memex system [Bush 1945] were to be built from transclusions, not links.

Project Xanadu

THE TRANSCOPYRIGHT LICENSE

ABOUT THE LICENSE.

Transcopyright is a copyright license intended to legitimize remix, "mashup," anthologies, and many other ways of combining content-- in any amount and in any combination.

The transcopyright license permits all parties to republish content indirectly, meaning that the new publisher doesn't send out the content, but rather the addresses of the content.

Any party in the universe may DOWNLOAD AND KEEP this content in any amount, provided that

- they obtain it from my designated server
- they retain it in a form connected to my designated server

Any party in the universe may REPUBLISH this content in any new context and any amount, provided that

- they republish only by giving the on-line addresses of the content on my designated server, for example by edit decision list (EDL).

CJEU: Posting a hyperlink to infringing content not a communication to public, unless...

October 26, 2016 · by marie-andree@maw-law.com ·

Is providing a hyperlink to a work protected by copyright, which was published online without the authorization of the right holder, an infringement of copyright under European Union law? In order to answer this question, the European Court of Justice (ECJ) had to decide whether providing such a hyperlink is a communication to the public within the meaning of Article 3(1) of Directive 2001/29 on the harmonisation of certain aspects of copyright and related rights in the information society (the InfoSoc Directive).

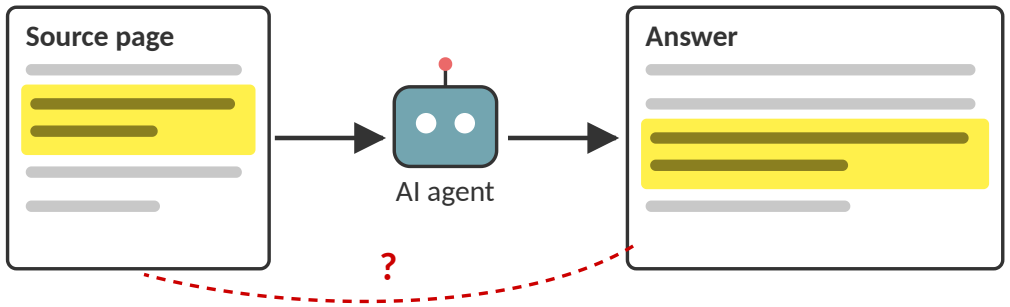
Article 3(1) provides that authors have the exclusive right to authorize or prohibit "any communication to the public of their works, by wire or wireless means, including the making available to the public of their works in such a way that members of the public may access them from a place and at a time individually chosen by them." The InfoSoc Directive does not define communication of the work to the public. Its Recital 23,

This issue was **highly debated** in the run-up to the **EU Copyright Directive** as recently as 2016!

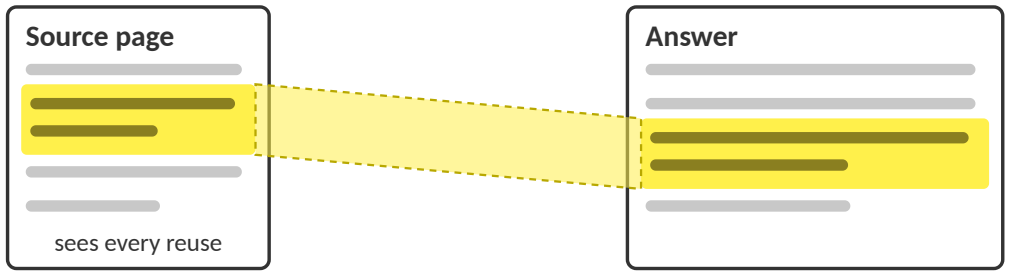
xanadu.com

Transclusion and LLM Agents

Web: the agent copies



Transclusion: the answer refers to the source



- **LLM agents** read Web pages and reuse their text in answers and actions
- On the Web, they **copy**: the answer loses its source, and the source sees the crawl but not the reuse
- With transclusion, each quote would **stay connected to its source**, with author and version
- Workarounds today: robots.txt opt-outs, **pay per crawl** with HTTP 402 Payment Required (Cloudflare, 2025)
- Not new: Ted Nelson coined **micropayments** for Xanadu, to pay the rights holders of every transcluded piece; the W3C later worked on payment requests in HTTP

What would AI agents gain or lose on a Web with transclusion?
Who would pay whom?

Cloudflare: Introducing pay per crawl, 2025; Wikipedia: Micropayment

Which is better - Xanadu or the Web?

We cannot say! What is “better” depends on the **targeted non-functional requirements**

Historically, the Web’s model has succeeded
But that’s a **one-sided experiment**, and history **is not over**

In honor of Ted Nelson, computer pioneers connect at 'Intertwingled' | Chapman Newsroom

> The Web did not evolve as he envisioned it and Nelson says he is both philosophical and disappointed. But he is confident his landmark Xanadu system will one day trump contemporary operating systems. As he told those gathered at the conference, “The world would have been a better place if I had succeeded, but I ain’t dead yet.”

Non-functional Profile of the World Wide Web

What we **can** today say is that the Web's Architecture is **doing (very) well** with some NFPs and **(very) poorly** with others

Case	Constraints	Desired NFPs	Trade-Offs
Lego	Uniform Interfaces (patented)	Interoperability, Extensibility	Design Freedom, Durability
Gothic Style	Pointed Arches, Flying Buttresses, Rib Vaults	Brightness	Cost, Aesthetics
Bologna	Uniform Interfaces (ECTS, BSc/MSc)	Interoperability, Scalability	Autonomy
Unix Philosophy	Uniform Interfaces (pipe-and-filter)	Interoperability, Extensibility	Design Freedom, Efficiency
Computer Networking	Layers, Uniform Interfaces	Extensibility, Adaptability	Performance
Flexible and Open World-Wide Information System		Scalability, Adaptability, Evolvability, Extensibility, Interoperability	Performance, Responsiveness, Data Integrity, Reliability

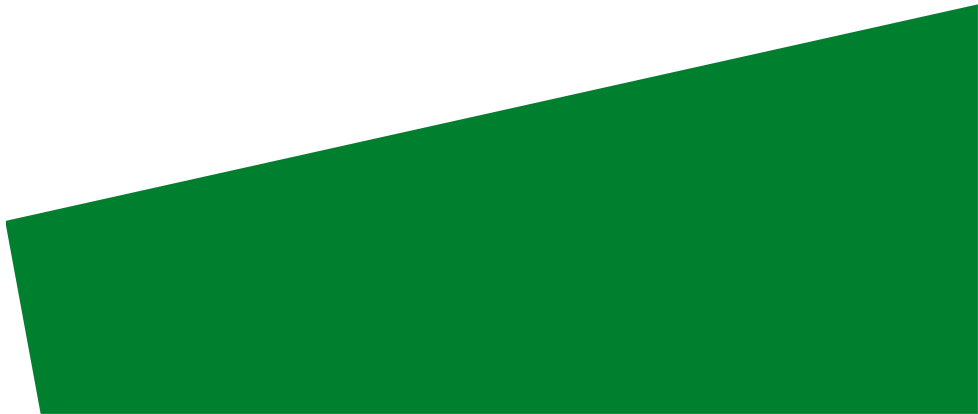
Architectural **Constraints?**

F. D. Davis. Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology, 1989 · D. Gefen, M. Keil. The Impact of Developer Responsiveness on Perceptions of Usefulness and Ease of Use, 1998 · D. Guinard, I. Ion, S. Mayer. REST or WS-*? A Developers' Perspective, 2011 · L. Popa, P. Wendell, A. Ghodsi, I. Stoica. HTTP: An Evolvable Narrow Waist for the Future Internet, 2012



Deriving the Web's architectural constraints

The REST Architectural Style



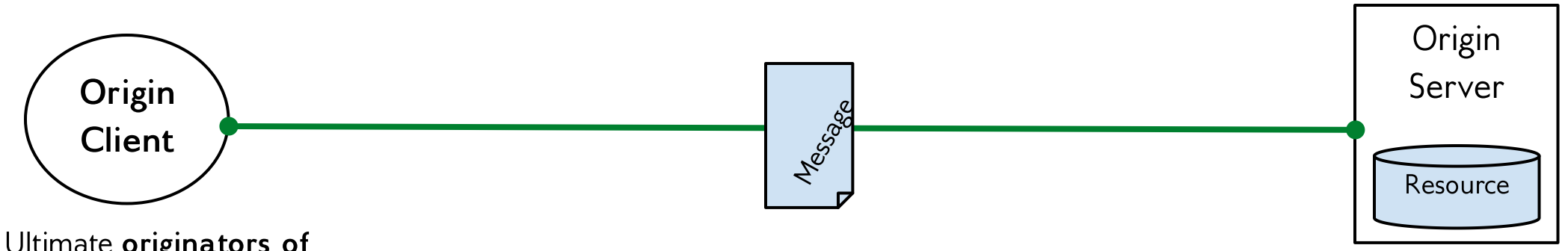
How would you construct a world-wide flexible and open information system?

Let's naturally derive the Web's core architectural constraints to get this profile:

- **Scalability** (in the sense of a global system)
- **Evolvability** (in the sense of decoupling between components)
- **Extensibility / Openness / Interoperability** (in the sense of heterogeneous service offerings and heterogeneous services partaking)

What architectural constraints do you propose?

We are of course talking about the REST architectural style here. If you know the REST constraints already, use this as an opportunity to give them a fresh look and to bring them together from an architectural viewpoint.



Ultimate **originators of requests** and ultimate recipients of responses

Ultimate **originators of responses** and ultimate recipients of requests

I will not cover the subsequent slides comprehensively. They are meant to accompany the required reading

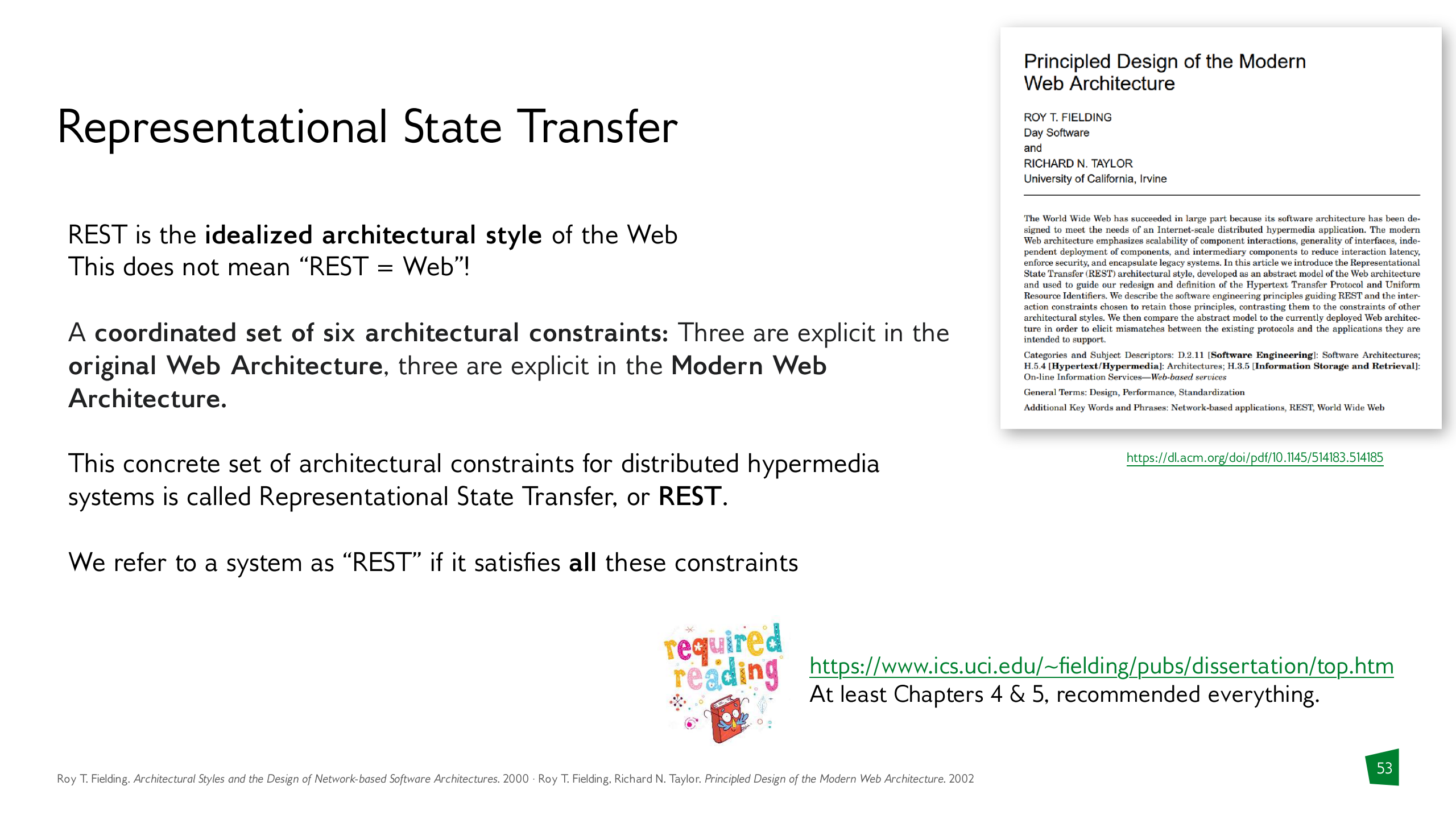
Representational State Transfer

REST is the **idealized architectural style** of the Web
This does not mean "REST = Web"!

A **coordinated set of six architectural constraints:** Three are explicit in the **original Web Architecture**, three are explicit in the **Modern Web Architecture.**

This concrete set of architectural constraints for distributed hypermedia systems is called Representational State Transfer, or **REST**.

We refer to a system as "REST" if it satisfies **all** these constraints



Principled Design of the Modern Web Architecture

ROY T. FIELDING
Day Software
and
RICHARD N. TAYLOR
University of California, Irvine

The World Wide Web has succeeded in large part because its software architecture has been designed to meet the needs of an Internet-scale distributed hypermedia application. The modern Web architecture emphasizes scalability of component interactions, generality of interfaces, independent deployment of components, and intermediary components to reduce interaction latency, enforce security, and encapsulate legacy systems. In this article we introduce the Representational State Transfer (REST) architectural style, developed as an abstract model of the Web architecture and used to guide our redesign and definition of the Hypertext Transfer Protocol and Uniform Resource Identifiers. We describe the software engineering principles guiding REST and the interaction constraints chosen to retain those principles, contrasting them to the constraints of other architectural styles. We then compare the abstract model to the currently deployed Web architecture in order to elicit mismatches between the existing protocols and the applications they are intended to support.

Categories and Subject Descriptors: D.2.11 [**Software Engineering**]: Software Architectures; H.5.4 [**Hypertext/Hypermedia**]: Architectures; H.3.5 [**Information Storage and Retrieval**]: On-line Information Services—*Web-based services*

General Terms: Design, Performance, Standardization

Additional Key Words and Phrases: Network-based applications, REST, World Wide Web

https://dl.acm.org/doi/pdf/10.1145/514183.514185

https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm
At least Chapters 4 & 5, recommended everything.

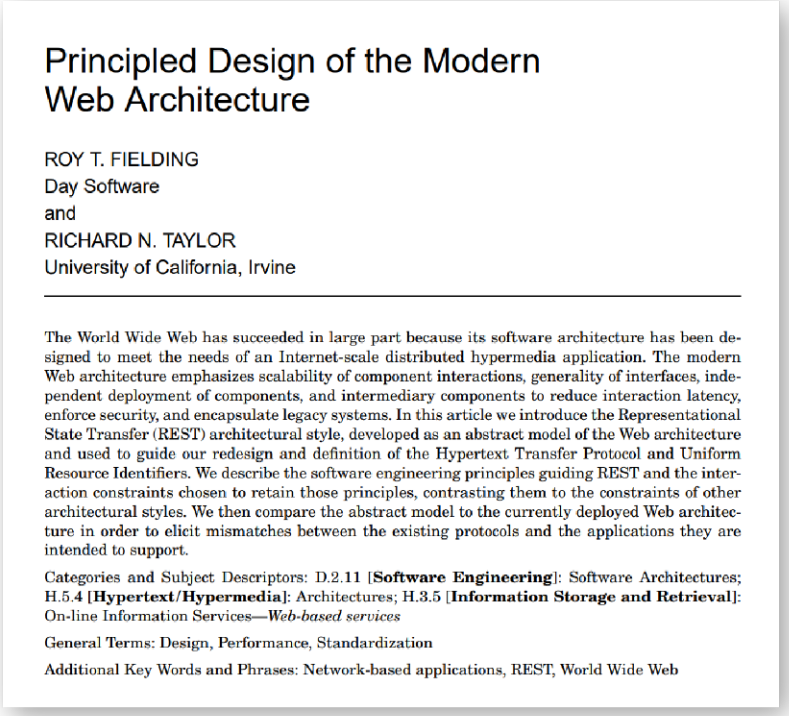
Representational State Transfer

"At the beginning of our efforts within the Internet Engineering Task Force (...) we recognized the need for a model of how the World Wide Web *should* work. This **idealized model of the interactions within an overall Web application** - what we refer to as the **Representational State Transfer (REST)** architectural style -became the foundation for the **modern Web architecture**, providing the guiding principles by which flaws in the existing architecture could be identified and extensions validated prior to deployment."

R.T. Fielding and R.N. Taylor, 2002

"Even if it were possible to build a software system that perfectly matches the requirements of its users, those requirements will change over time just as society changes over time. A long-lived system like the Web must be prepared for (...) gradual and fragmented change, where old and new implementations co-exist without preventing the new implementations from making use of their extended capabilities. (...) The introduction of each new goal presents us with a challenge**: how do we introduce a new set of functionality to an architecture that is already widely deployed, and how do we ensure that its introduction does not adversely impact, or even destroy, the architectural properties that have enabled the Web to succeed? It was this question that motivated our development of the REST architectural style."**

R.T. Fielding and R.N. Taylor, 2002



Principled Design of the Modern Web Architecture

ROY T. FIELDING
Day Software
and
RICHARD N. TAYLOR
University of California, Irvine

The World Wide Web has succeeded in large part because its software architecture has been designed to meet the needs of an Internet-scale distributed hypermedia application. The modern Web architecture emphasizes scalability of component interactions, generality of interfaces, independent deployment of components, and intermediary components to reduce interaction latency, enforce security, and encapsulate legacy systems. In this article we introduce the Representational State Transfer (REST) architectural style, developed as an abstract model of the Web architecture and used to guide our redesign and definition of the Hypertext Transfer Protocol and Uniform Resource Identifiers. We describe the software engineering principles guiding REST and the interaction constraints chosen to retain those principles, contrasting them to the constraints of other architectural styles. We then compare the abstract model to the currently deployed Web architecture in order to elicit mismatches between the existing protocols and the applications they are intended to support.

Categories and Subject Descriptors: D.2.11 [**Software Engineering**]: Software Architectures; H.5.4 [**Hypertext/Hypermedia**]: Architectures; H.3.5 [**Information Storage and Retrieval**]: On-line Information Services—*Web-based services*

General Terms: Design, Performance, Standardization

Additional Key Words and Phrases: Network-based applications, REST, World Wide Web

https://dl.acm.org/doi/pdf/10.1145/514183.514185

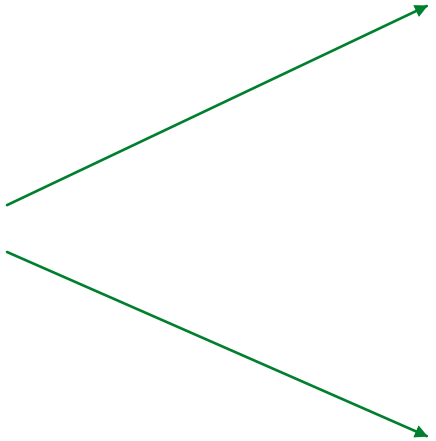
Architectural Constraints on the Modern Web Architecture

Similar to how Lego restricts interfaces and the Bologna Reform restricts study programs, Representational State Transfer (REST) **restricts how components may process requests and send responses.**

Six architectural constraints to gain desirable non-functional properties

- Three of these were **part of the pre-1994 Web architecture**
- Three were added to guide the design of the **modern Web architecture**

Together, they continue to make the Web an **Internet-scale**, **open**, and **long-lived** system



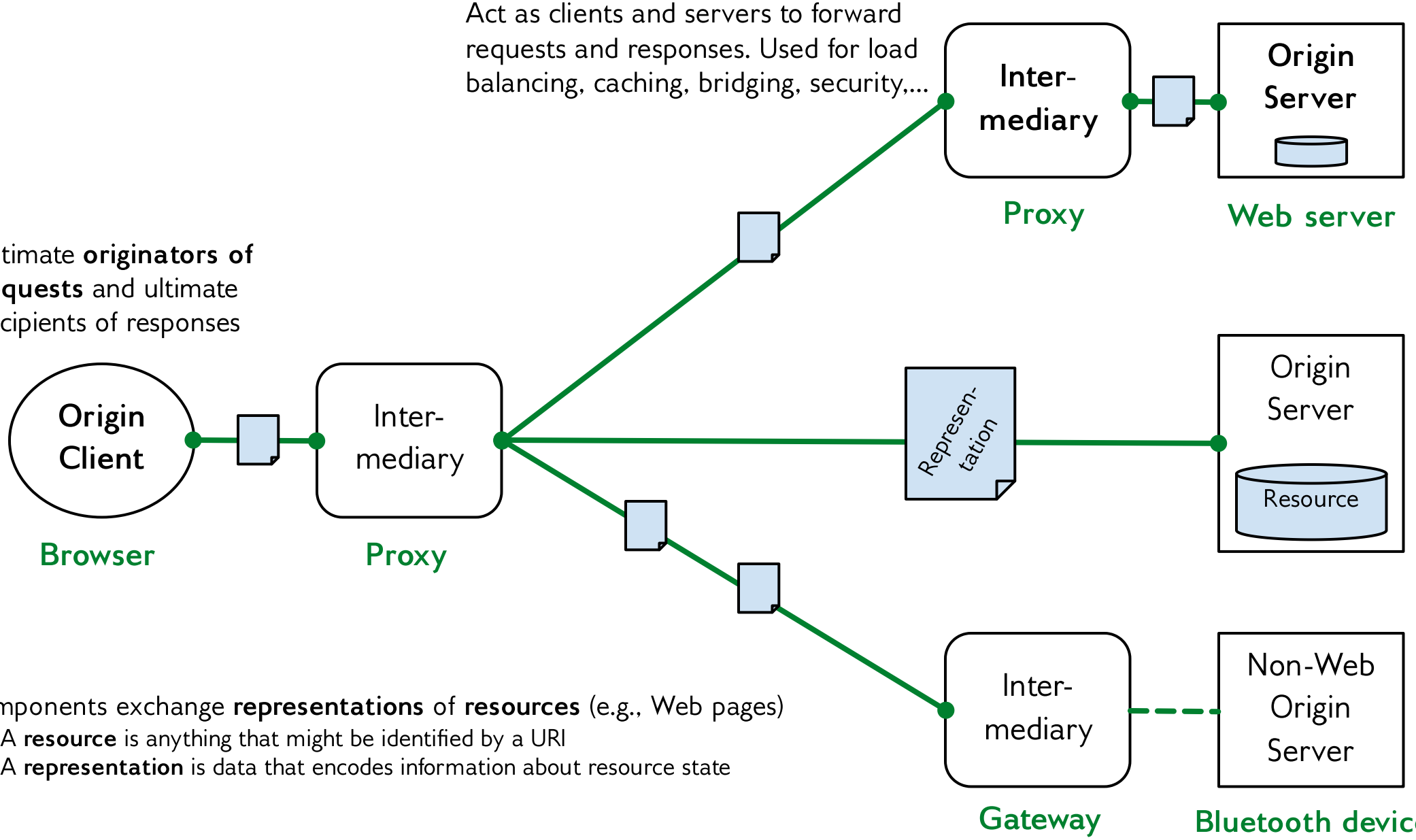
- Client-Server
- Statelessness
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- Uniform Interface

What follows is an **illustrated summary to accompany your required reading**

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Act as clients and servers to forward requests and responses. Used for load balancing, caching, bridging, security,...

Ultimate **originators of requests** and ultimate recipients of responses



Components exchange **representations** of **resources** (e.g., Web pages)

- A **resource** is anything that might be identified by a URI
- A **representation** is data that encodes information about resource state

Client-Server

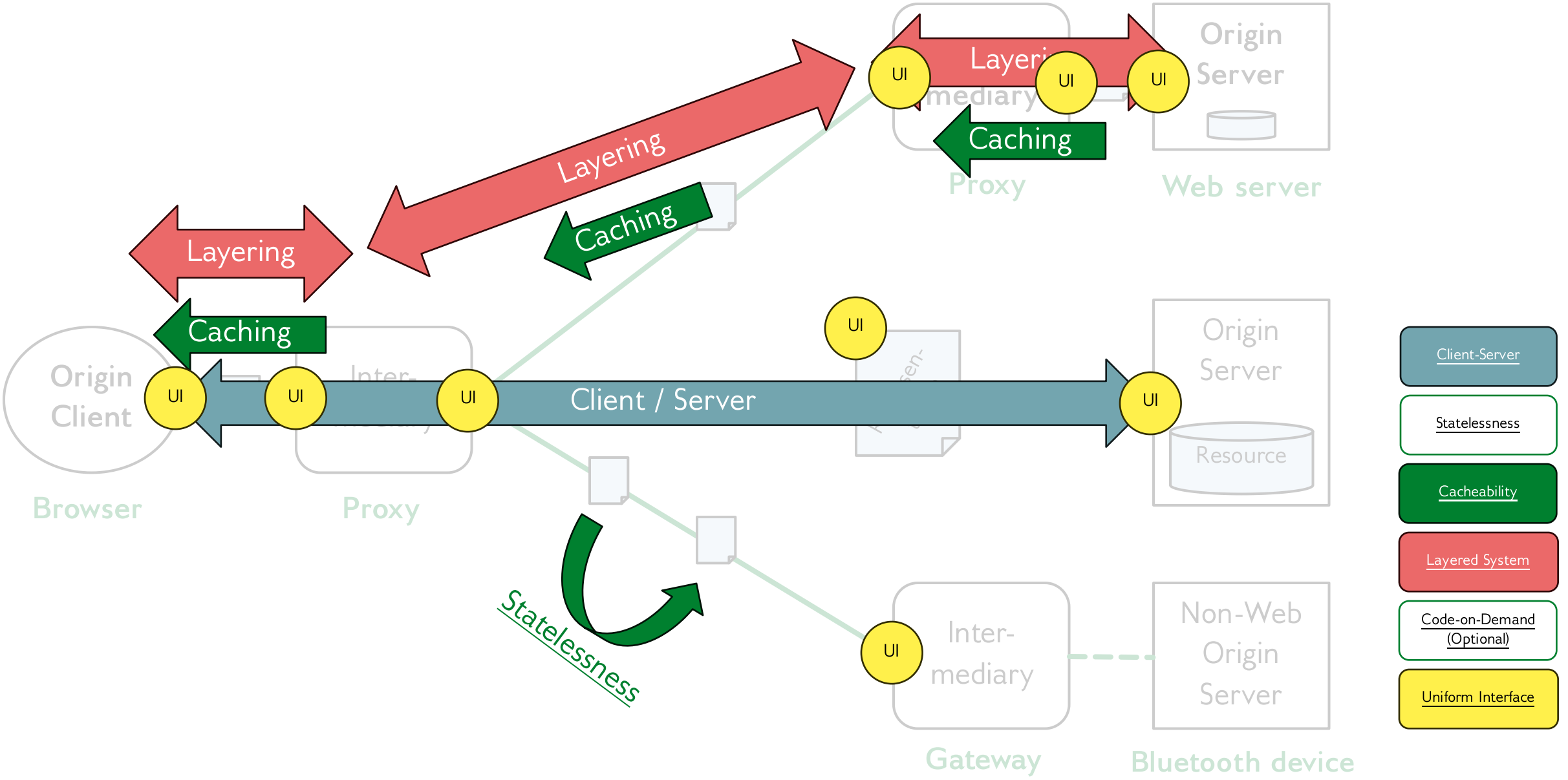
Statelessness

Cacheability

Layered System

Code-on-Demand (Optional)

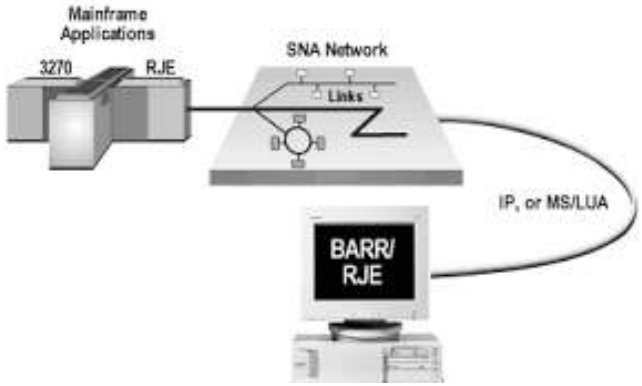
Uniform Interface


Origin Server
Proxy
Web server
Layering
Caching
Layering
Caching
UI
UI
UI
Caching
Origin Client
Browser
Inter-
Proxy
UI
UI
UI
Client / Server
UI
Origin Server
Resource
Statelessness
UI
Inter-mediary
Gateway
Non-Web Origin Server
Bluetooth device
Client-Server
Statelessness
Cacheability
Layered System
Code-on-Demand (Optional)
Uniform Interface

Client-Server: Three High-level Options

Traditional Client/Server Style

- Render data where it rests
- **Move rendered result** to client in **fixed format**
- Great simplification of clients!
- Server-side scalability problems (large load on server)



Mobile Object Style

- Encapsulate data with rendering engine; **move both** to the recipient
- Great information hiding! Data format does not matter
- Limits functionality to what is foreseen in the engine; transmits a lot of data

Prenegotiated Types Style

- Send raw data to recipient together with metadata that describes it
- Remains scalable (load sharing between client and server)
- Loses information hiding; requires prenegotiated data types

Client-Server

Statelessness

Cacheability

Layered System

Code-on-Demand (Optional)

Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

These: https://www.iana.org/assignments/media-types/

What's meant here?

Client-Server

REST Client-Server: Hybrid Style

- Shared understanding of data types with metadata but **limited to a standardized interface**
- Approximation of Mobile Object Style: Interaction by **transferring representations** according to an evolving set of dynamically selected standard data types
- Gains **separation of concerns** (e.g., UI vs. Data Storage) between **clients** (that send requests) and **servers** (that receive requests and may send responses)

Non-functional benefits

+ evolvability (independent evolution of components through SoC)

+ scalability (on the server side)

+ portability (of user interfaces: same user interface across platforms)

Non-functional drawbacks

- **performance** (cannot optimize across client and server)
- **responsiveness** (cannot optimize across client and server)
- **behavioral consistency** (different clients may behave differently)

Client-Server

Statelessness

Cacheability

Layered System

Code-on-Demand (Optional)

Uniform Interface

Stateless Interactions

Each client-server interaction is **self-contained** and **stands alone**

State is kept in resources and in the client, but **not in the interaction**

Resource state is managed on the server
- It is the same for every client working with the resource
- When a client modifies the resource state, other clients see this change as well
- The server does not remember the requesting client (no session) nor previous requests (no history)

Application state is managed on the client
- It is specific for a client and thus has to be maintained by each client
- On the Web, this is the purpose of “Cookies”

Client-Server

Statelessness

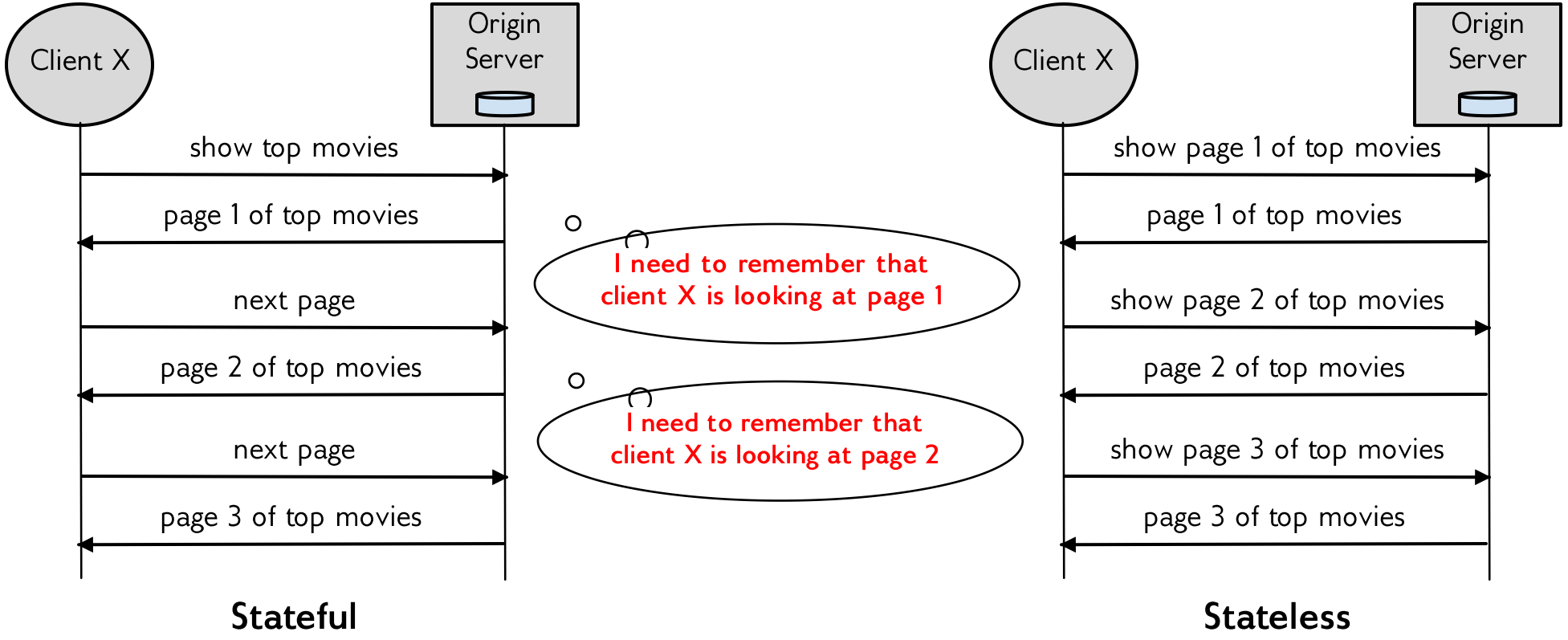
Cacheability

Layered System

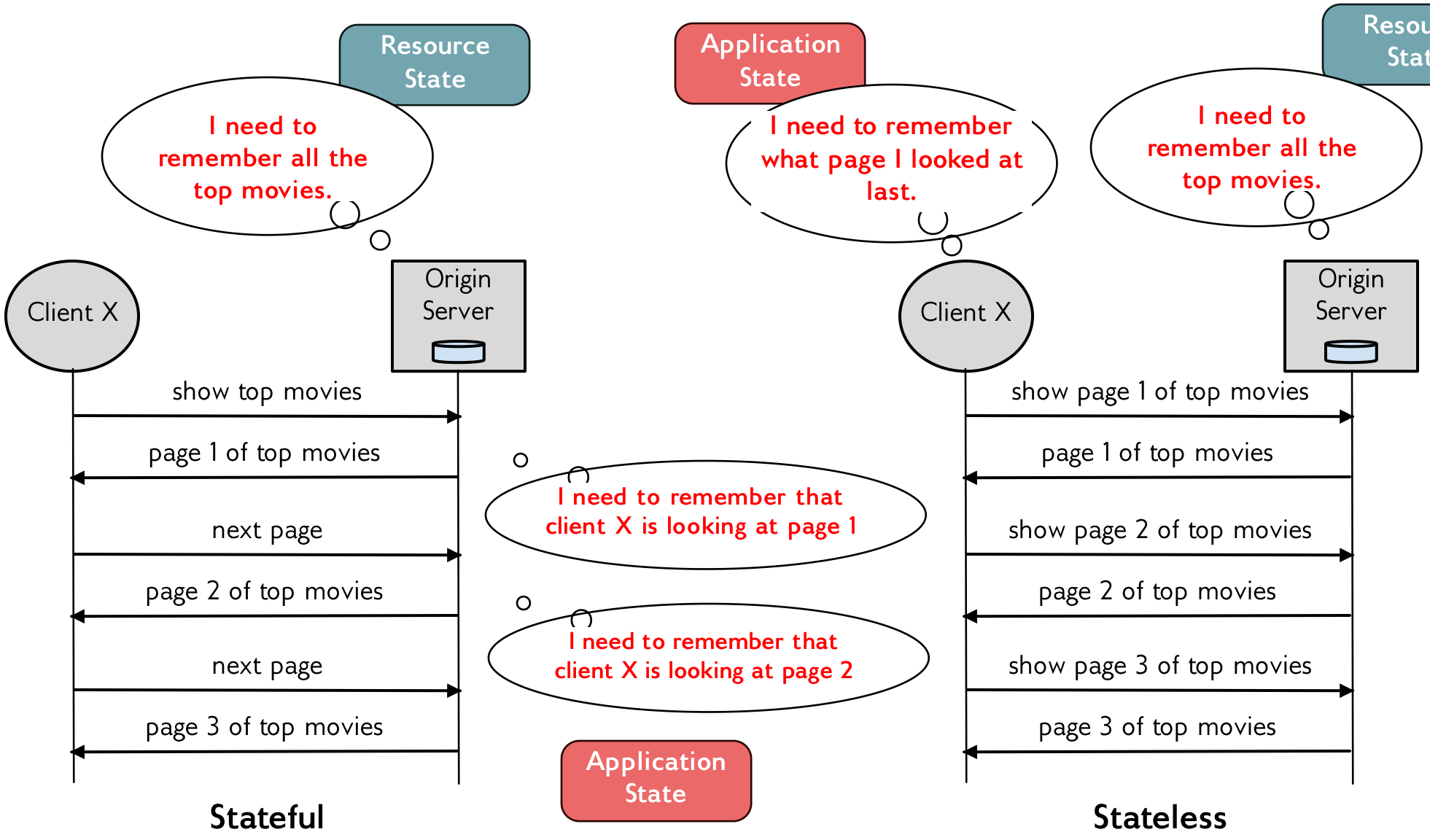
Code-on-Demand (Optional)

Uniform Interface

Stateless Interactions



- Client-Server
- **Statelessness**
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- Uniform Interface



- Client-Server
- **Statelessness**
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- Uniform Interface

Stateless Interactions

Non-functional benefits

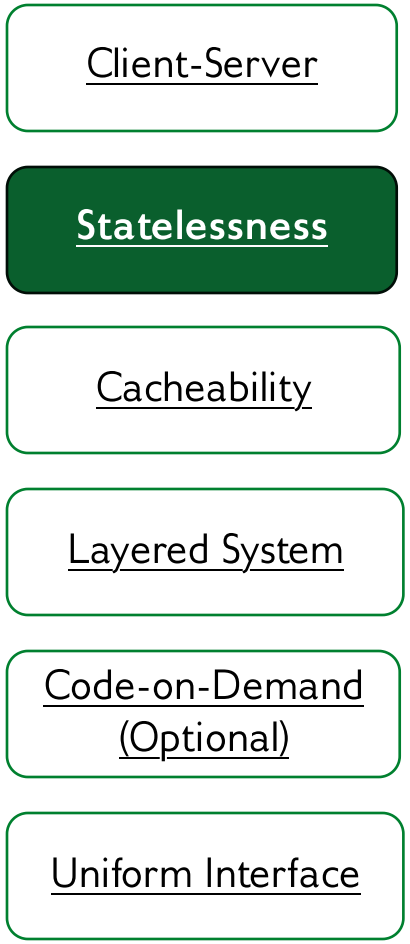
+ **reliability** (easier to recover interaction from partial failures)
+ **scalability** (takes load off servers, no management of interactions)
+ **visibility** (easy monitoring)
+ **load balancing**

Non-functional drawbacks

- **performance** (repeated transmission of the same data)
- **responsiveness** (more complex internal server logic)
- **consistency** (less server control over consistent application behavior)

Practically

- One origin server can easily serve many clients
- Intermediaries can reliably interpret requests
- Requests can more easily be distributed across different origin servers (load balancing)



Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Cacheability

This response is **fresh** for **604800 seconds** since it was **generated on the origin server** and has already been cached for a total of **440634 seconds**

ETag identifies a **version** of the resource (used to **save bandwidth** if a resource is unchanged and to **prevent concurrent updates**)

The response expires after this date (ignored if Cache-Control with max-age is set)

This response was returned from a **cache**

```
  Trying 93.184.215.14:80...
Connected to example.org (93.184.215.14) port 80
GET / HTTP/1.1
Host: example.org
User-Agent: curl/8.3.0
Accept: */*

HTTP/1.1 200 OK
Age: 440634
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Tue, 01 Oct 2024 21:06:50 GMT
Etag: "3147526947+gzip+ident"
Expires: Tue, 08 Oct 2024 21:06:50 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECAcc (bsb/2794)
Vary: Accept-Encoding
X-Cache: HIT
Content-Length: 1256

<!doctype html>
<html>
<head>
    <title>Example Domain</title>
```

- Client-Server
- Statelessness
- **Cacheability**
- Layered System
- Code-on-Demand (Optional)
- Uniform Interface

Cacheability

Servers **must indicate** whether their responses **may be cached**.

Non-functional benefits

+ performance (on the network – eliminates interactions)
+ responsiveness (as perceived by the user – eliminates interactions or pre-fetches information)
+ scalability (reduced load on server)

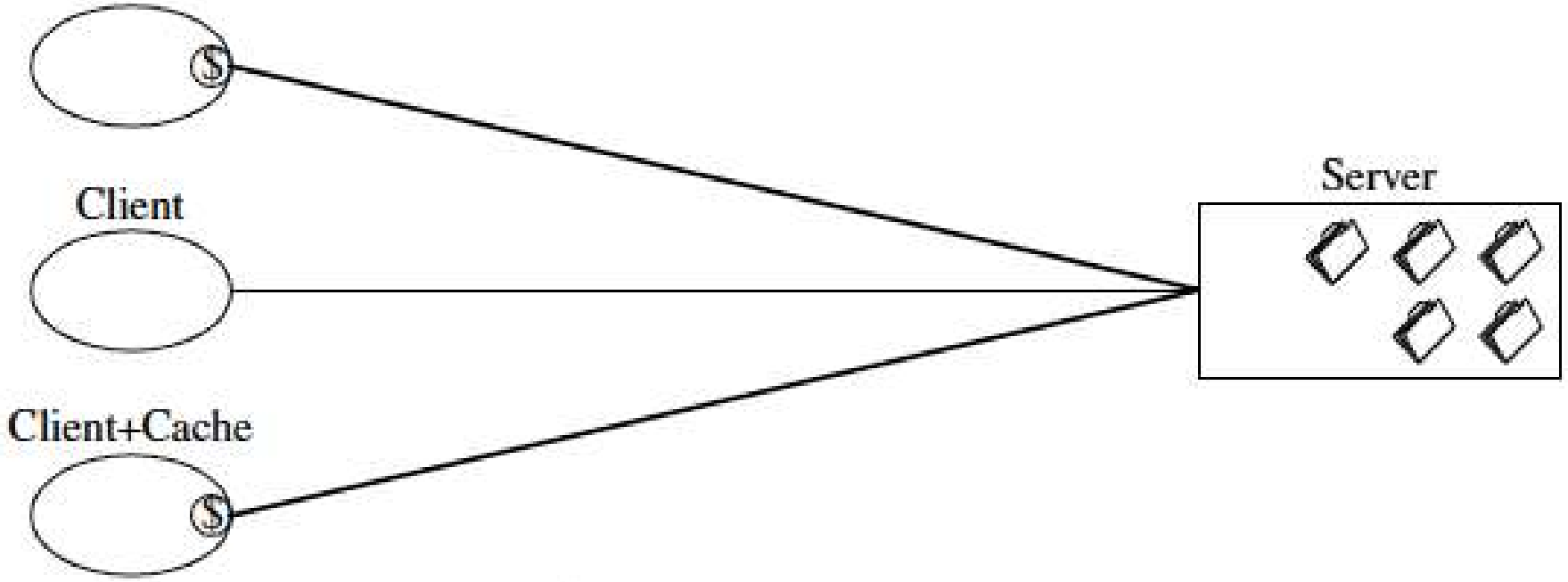
Non-functional drawbacks

- **reliability** (stale data)

Client-Server
Statelessness
Cacheability
Layered System
Code-on-Demand (Optional)
Uniform Interface

The Early Web Architecture (before 1994)

(founded in 1994)



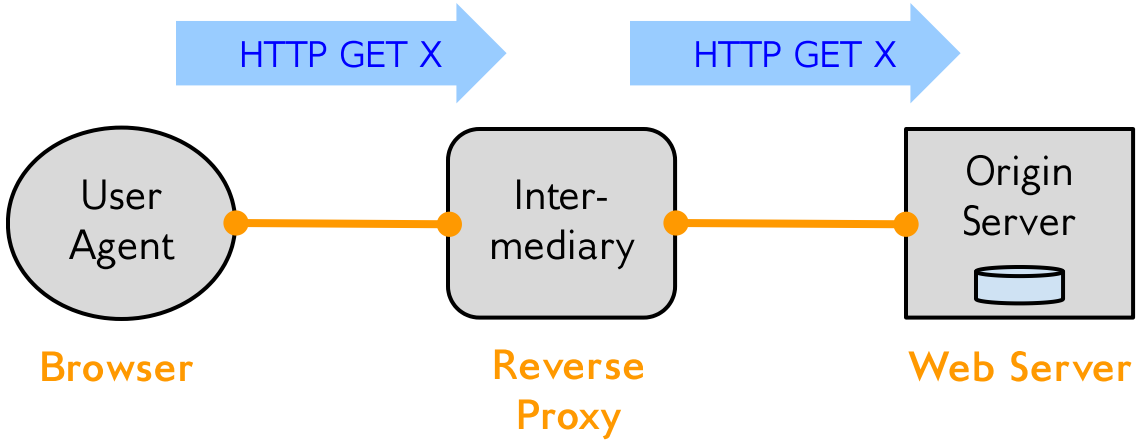
- Client-Server
- Statelessness
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Layered System

Further constrains client-server by **restricting visibility to a single component layer**

Components are oblivious of whether they communicate with origin servers/clients or with intermediaries

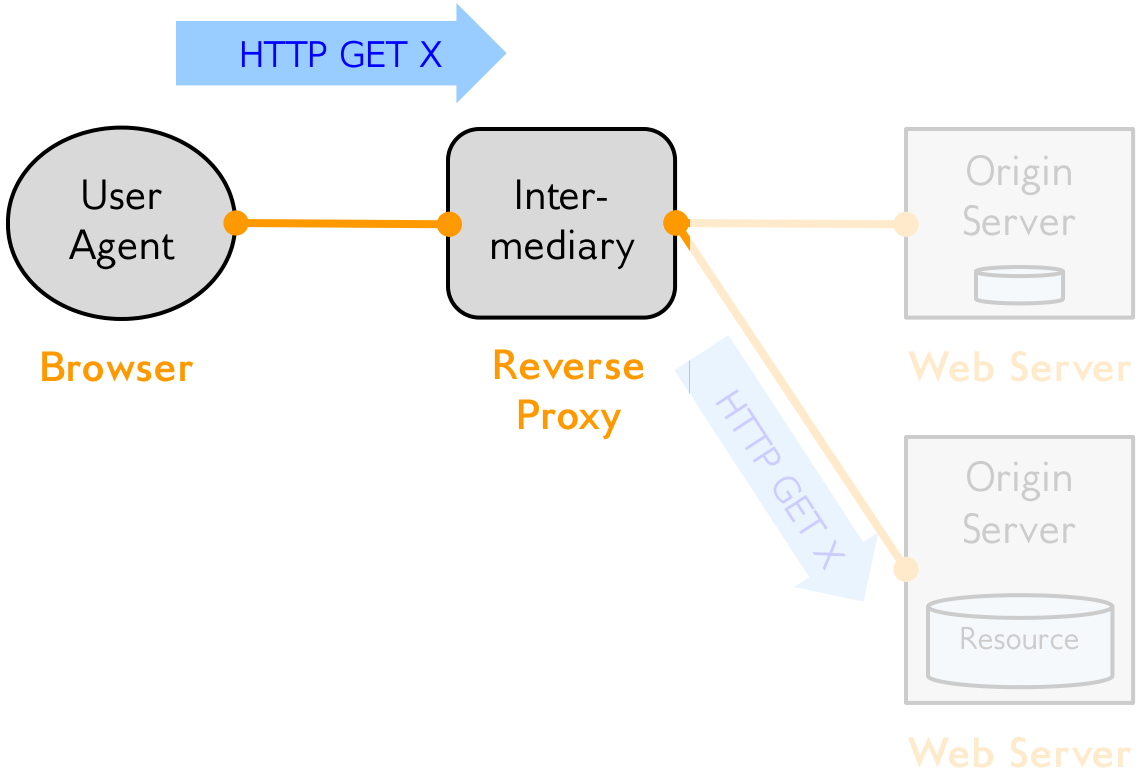


- Client-Server
- Statelessness
- Cacheability
- **Layered System**
- Code-on-Demand (Optional)
- Uniform Interface

Layered System

Further constrains client-server by **restricting visibility to a single component layer**

Components are oblivious of whether they communicate with origin servers/clients or with intermediaries



What standard software design principle is this?

- Client-Server
- Statelessness
- Cacheability
- **Layered System**
- Code-on-Demand (Optional)
- Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Layered System

Further constrains client-server by **restricting visibility to a single component layer**
Components are oblivious of whether they communicate with origin servers/clients or with intermediaries

Non-functional benefits
+ scalability (scope restriction for components / information hiding)

Flight of the Starlings: Watch This Eerie but Beautiful Phenomenon | Short Film Showcase

Non-functional drawbacks
- **performance** (user-perceived as well as network overhead due to many indirections / marshalling)

Client-Server
Statelessness
Cacheability
Layered System
Code-on-Demand (Optional)
Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Code-on-Demand (Optional)

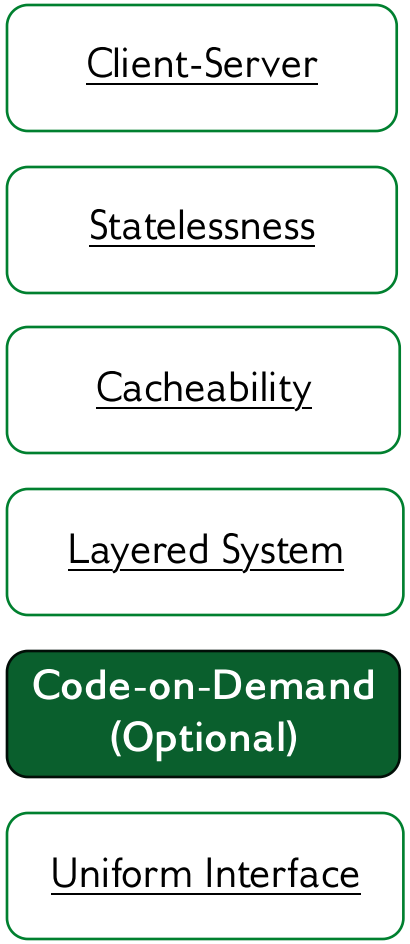
Component functionality can be **extended by moving and executing code** (e.g., scripts).

Non-functional benefits

+ extensibility (client behavior can be enriched at run time)

Non-functional drawbacks

- **maintainability** (through reduced visibility)
- **security** (client needs to execute ad-hoc received code)



Uniform Interface



Component implementations are **heterogeneous** (different frameworks, data models, formats, etc.). How can they **interact in a uniform manner?**

Components interact through a **general**, **uniform interface** that hides away their implementation details!

Analog: Interaction via a **shared language** (often: EN)



Individuals can now **interact and understand** one another ("interoperability")

However, the "uniform interface" is a compromise

- Might be a **reduced** interface (i.e., loss of expressivity)
- Might be an **inefficient** interface (e.g., for non-native English speakers)

Client-Server
Statelessness
Cacheability
Layered System
Code-on-Demand (Optional)
Uniform Interface

Uniform Interface



Components interact through a **general**, **uniform interface** that hides away their implementation details!

Extension on top of *Layered System* to **reduce coupling between components**.

The same (small) set of **verbs** (i.e., operations) applies to every **noun** (i.e., resource)

- Verbs and their semantics are **universal** and are **not invented per-application**
- New **verbs rarely** enter language; new **nouns regularly** do
- The same pattern is true for the "English" uniform interface, see Oxford's New Word Entries

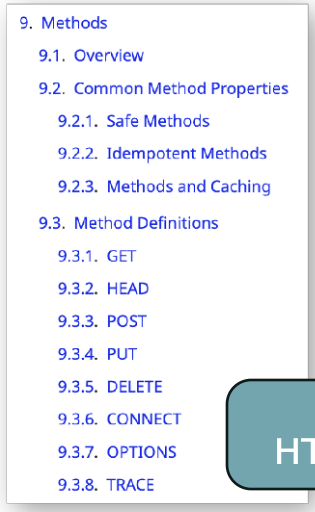
For protocols that follow the REST architectural style...

...**Noun semantics** are defined by **media types** (e.g., application/json for JSON)

...**Verb semantics** are defined in the **protocols** (e.g., in HTTP)

The methods apply to **every resource**

The methods have **global semantics**



RFC 9110: HTTP Semantics

- Client-Server
- Statelessness
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- **Uniform Interface**

HTTP's Uniform Interface

GET: retrieves the representation of a resource
HEAD: identical to GET but server must not send content
OPTIONS: request information about communication options of resource
PUT: replaces the state of a resource with the one enclosed in the body of the request
PATCH (added through RFC 5789): apply specified changes to resource
DELETE: removes the target resource
POST: wildcard method, typically used for creating or updating a resource

safe, idempotent, cacheable

idempotent

neither safe nor idempotent

Client-Server

Statelessness

Cacheability

Layered System

Code-on-Demand (Optional)

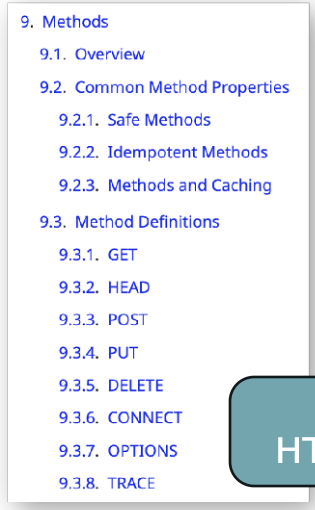
Uniform Interface

Some notes on GET requests:

- GET requests can be cached
- GET requests remain in the browser history
- GET requests can be bookmarked
- GET requests should never be used when dealing with sensitive data
- GET requests have length restrictions
- GET requests are only used to request data (not modify)

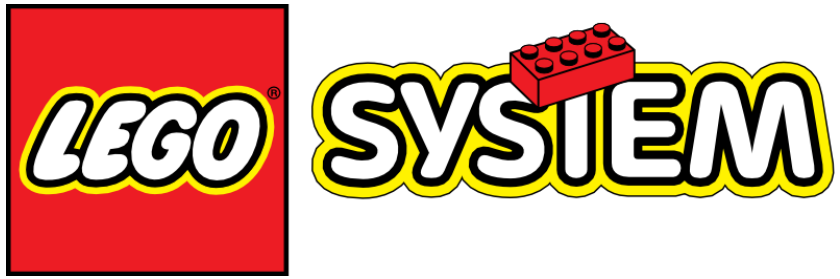
Some notes on POST requests:

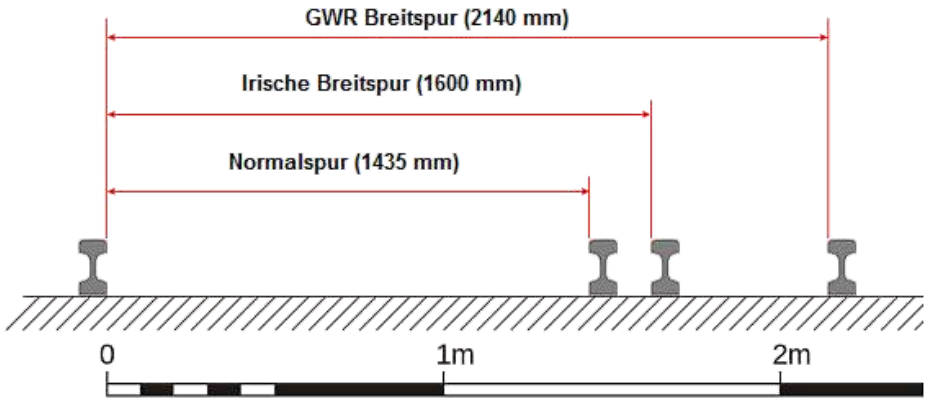
- POST requests are never cached
- POST requests do not remain in the browser history
- POST requests cannot be bookmarked
- POST requests have no restrictions on data length



RFC 9110: HTTP Semantics

Uniform Interface Trade-Offs





Non-functional benefits

+ evolvability / extensibility / interoperability (permits independent evolution of components)
+ maintainability / visibility (easier to monitor interactions)
+ simplicity

Non-functional drawbacks

- **performance** (general uniform interface is not optimized for specific applications)

Client-Server

Statelessness

Cacheability

Layered System

Code-on-Demand (Optional)

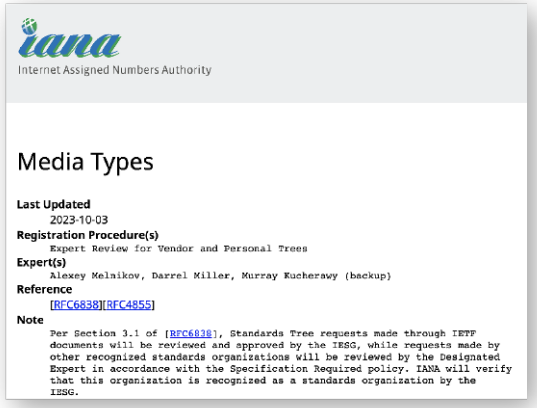
Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

What aspects are “uniform”?

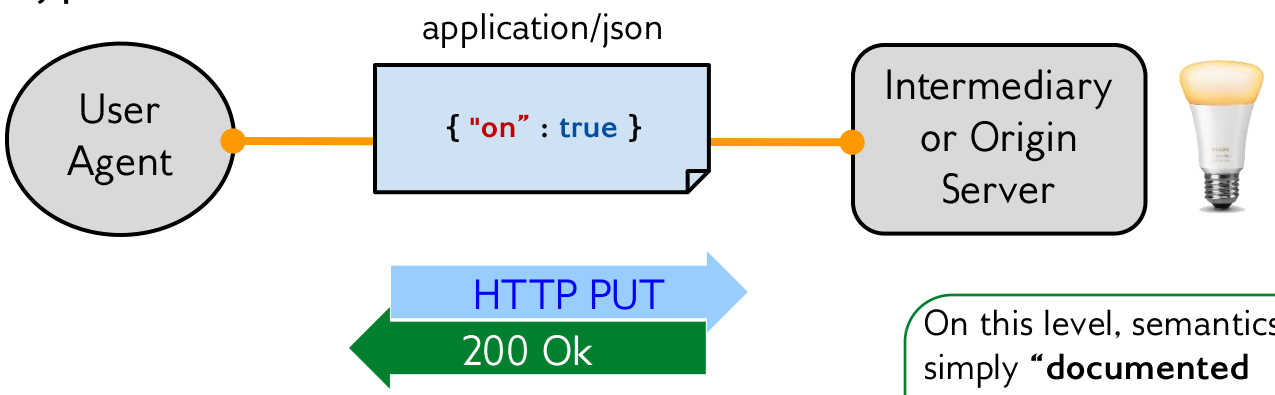
The **interface itself**: There is only a small set of general methods (e.g., GET, POST, etc.) that apply to any resource

The **transmitted representations**: Their semantics are defined using media types.



Does this really specify the **semantics** of the interaction?

No...



So what now?

We could agree on a sub-type to define the intended semantics, e.g. **application/bulb+json**

On this level, semantics are simply **“documented consensus among people”**

This consensus is then **programmed into a machine.**

- Client-Server
- Statelessness
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- **Uniform Interface**

More in 3 weeks

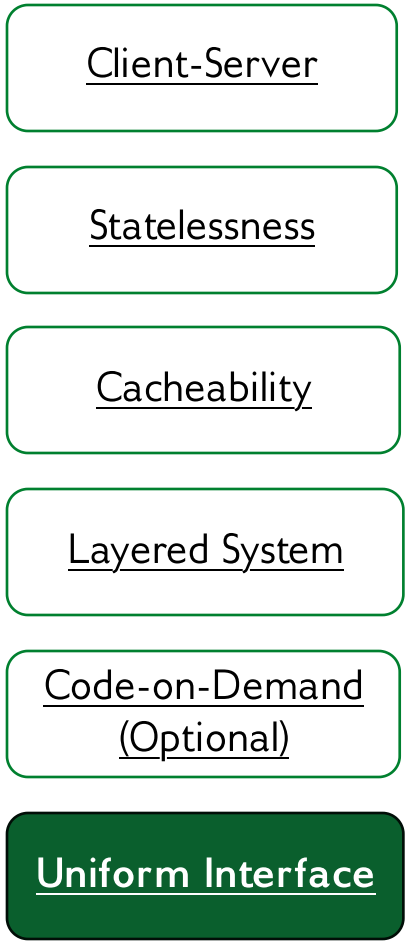
REST's 4 Uniform Interface Constraints

1. Resources must have **uniform identifiers** that are uniquely addressable using a universal syntax

2. There is **no direct access** to resources. Rather, components exchange representations of resources and manipulate resources via representations.

3. Messages must be self-descriptive. They need to include **metadata** that describes their meaning.

4. The state of the client-server application is **guided by server-provided hyperlinks**.



Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Pointer Lecture 8: URIs and Statelessness in ROA / CoH ↗

(1/4) Uniform Identification

Name everything you want to talk about in your application. Those are resources.

- **Products** in an online shop
- **Customers** that are expected to buy products
- **Shopping Carts** where customers collect products
- **Next** links on multi-page submission processes or **paged results**
- **Robots** in a laboratory



The standard RFC 3986 „URI“ defines the term “resource” as **anything** that might be referred to by a URI.

- **Abstract** concepts can also be resources
- **Non-network-retrievable** artifacts can also be resources

`https://github.com/smnmyr#me`

Resources have uniform identifiers that are uniquely addressable using a universal syntax

- They should mean the same anywhere (think load balancing, but also intranets or DNS spoofing)

Client-Server
Statelessness
Cacheability
Layered System
Code-on-Demand (Optional)
Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Pointer Lecture 8: From WS-* to CRUD-over-HTTP (CoH) ↗

(2/4) Manipulation via Representations

Resources are **abstract** entities and cannot be used per se

Interaction with resources happens via **resource representations**

- Clients don't have access to resources but exchange **representations** with servers
- It is **communicated** which kind of representation is used – content negotiation establishes a **contract**
- Representation formats can be **negotiated between peers**

When you access a Web resource, you see **one of its representations**

- Your browser **negotiates** for you! (HTML, locale/language, context attributes)
- If a machine accesses a Web resource instead of a person, it will (should!) see a **different representation**

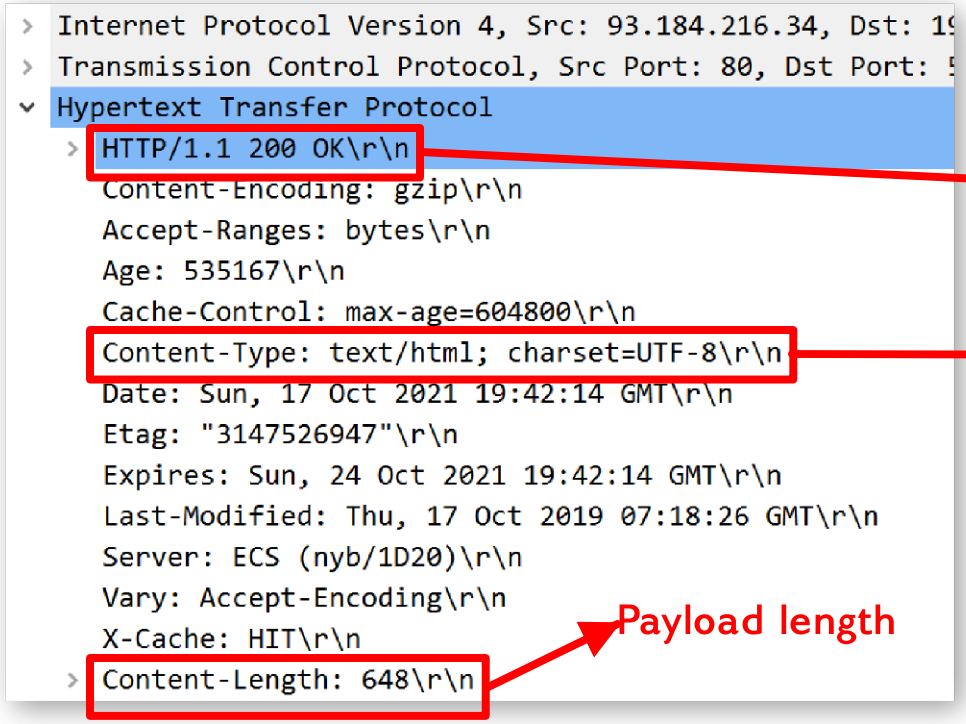
Client-Server
Statelessness
Cacheability
Layered System
Code-on-Demand (Optional)
Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Pointer Lecture 9: Media Types + RDF Vocabularies ↗

(3/4) Self-descriptive Messages

A message (e.g., HTTP request, CoAP response) must include **metadata** that describes its meaning; this includes standard verbs and media types



Response **status code** (and HTTP version)

Media type of the response

+ Cache control metadata, etc.

Clients, Servers, and Intermediaries can interpret messages (e.g., to help the Web scale up)

- Client-Server
- Statelessness
- Cacheability
- Layered System
- Code-on-Demand (Optional)
- **Uniform Interface**

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

(4/4) Guidance through Server-Provided Hyperlinks

In REST, servers give local guidance to clients through hyperlinks
Hyperlinks are **possible state transitions of the client/server application**

Clients don't "call services", they **"navigate application states"**

- In REST, clients **discover** hyperlinks at **run time!**
- Representations contain information about **possible traversals**
- The application navigates to the next resource depending on **link semantics**
- Navigation **can be delegated** since interactions are **uniform**

This "simple" mechanism renders the Web **robust and flexible**

- This is why Web browsers **are general-purpose tools** and not hard-coded to specific applications!
- This enables clients to **automatically adapt** when hyperlinks change! They don't even notice!

"HATEOAS" (Hypermedia as the Engine of Application State) is a great way of creating **highly flexible and interoperable systems** beyond the Web!
A lot more about this in Lecture 9 and Assignment 9.

Client-Server

Statelessness

Cacheability

Layered System

Code-on-Demand (Optional)

Uniform Interface

Roy T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000 · Roy T. Fielding, Richard N. Taylor. *Principled Design of the Modern Web Architecture*. 2002

Summary: Evaluating „Styles“

Actually, an architectural **style**...

gRPC vs REST

TECH SCHOOL

FEATURE	GRPC	REST
Protocol	HTTP/2 (fast)	HTTP/1.1 (slow)
Payload	Protobuf (binary, small)	JSON (text, large)
API contract	Strict, required (.proto)	Loose, optional (OpenAPI)
Code generation	Built-in (protoc)	Third-party tools (Swagger)
Security	TLS/SSL	TLS/SSL
Streaming	Bidirectional streaming	Client → server request only
Browser support	Limited (require gRPC-web)	Yes

https://dev.to/techschoolguru/is-grpc-better-than-rest-where-to-use-it-3blg

Frank Szendzielarz • Aug 10 '20

I am not sure I understand the comparison.

REST is an architectural principle. REST is about various things, such as representing a set of resources with a common interface, for example the HTTP verb set. REST doesn't care, I think, about the serialisation format or the transport.

How is gRPC even comparable to REST? Are you trying to compare gRPC with HTTP + JSON media formatters? Or is the intention something else?

Apologies if I have misunderstood the article.

1 like Reply

TECH SCHOOL • Aug 10 '20

Hi Frank, you're right, I actually meant to compare gRPC vs normal HTTP JSON API.

2 likes Reply

Towards Assignment 7

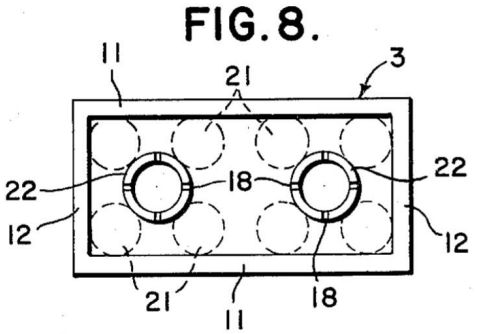
Task	What you do	On the slides
1 Characterization	read your role briefing; your objectives, what to avoid, and which features of the standard follow from that	
2 Uniform Interaction with HTTP	2.1 integrate the Auction House into your PITAS application; 2.2 define a uniform HTTP API for it; ADRs for your decisions	Uniform interface, identification
3 PITAS Interoperability Task Force	one delegate per group; agree on identifiers, HTTP methods and media types; keep the decision log	Self-descriptive messages, hypermedia
4 Reflecting on Your Role	which of your objectives G1 to G5 are on track, which are at risk	

Deadline: Monday 16.11.2026, 23:59 CET. Hand in a GitHub release with your ADRs, and a PDF with Tasks 1 and 4.
PITAS Plugtest: Tuesday 24.11.2026, 17:00 to 20:00.



Summary: Towards Assignment 7

With Uniform Interfaces, we **hide component implementation details** behind standardized interfaces.
Among others, this **improves interoperability** but **constrains design freedom.**



In Assignment 7, you will need to define a **uniform interface** across groups

- How **deep of a binding** do you introduce in your system?
- Which **aspects** do you standardize and what do you leave open?

Identifiers (e.g., URIs)?
Interaction model (e.g., HTTP verbs)?
Representation format (i.e., media supertypes)?
Representation schema and semantics (i.e., media subtypes)?
Data within the transmitted representations?

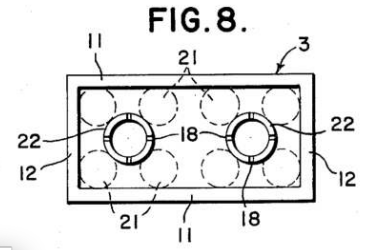
Can you **re-use** already existing standards? If not, can you **extend** existing standards?
Do look at **existing** media types and method semantics – others **probably already solved a similar problem**!

Summary: Towards Assignment 7



With Uniform Interfaces, we **hide component implementation details** behind standardized interfaces. Among others, this **improves interoperability** but **constrains design freedom.**

Take care and be realistic: **Technology is not everything in such discussions**!



Example: QUIC-Multipath

> The present charter is silent on multi-path, see:
> https://datatracker.ietf.org/wg/quic/about
>
> The conclusion of rechartering that happened last year, led to this conclusion, see:
> https://mailarchive.ietf.org/arch/msg/quic/rcPf7u9AHIGwNr6j0ZqrqFujVvk
>
> Not sure there is a source that outlines it was political - I suppose would lead to bad press for one of the camps, but one can read between the lines ;)

Next steps for Multipath QUIC (IETF QUIC mailing list)

> Hi Simon,
>
> you're right.
>
> ISPs, network vendors and academics are still very much interested in multi-path, and this camp is not insignificant either, so there is hope.
>
> Even if some hyper-giants decide not to offer multi-path capability on the phones, perhaps there could be other verticals where multi-path can see traction.

RFC 10069: Managing Multiple Paths for a QUIC Connection

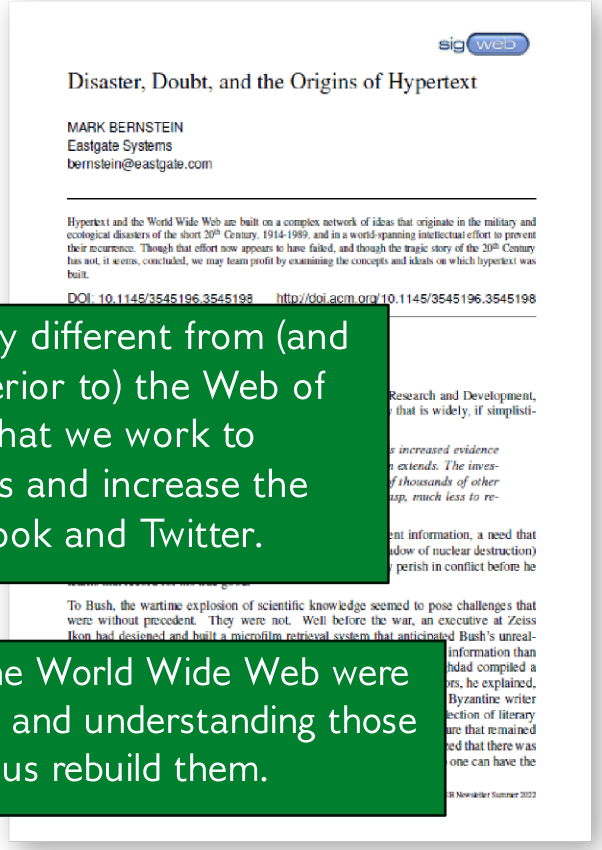
Summary: Towards Assignment 7



Set aside time for the required readings.

- Fielding and Taylor: Principled Design of the Modern Web Architecture. ACM Transactions on Internet Technology 2(2), 2002
- Bernstein: Disaster, Doubt, and the Origins of Hypertext. ACM SIGWEB Newsletter, 2022
- Bekavac and Mayer: Broken Links and Fading Stories: How Dominant Online Platforms Subvert the Hypermedia Foundation of the Web. WebSci 2026
- Webber and Parastatidis: REST in Practice (Chapters 1-5; over the next 4 weeks)
- Berners-Lee: Information Management: A Proposal. CERN, 1989





sigweb

Disaster, Doubt, and the Origins of Hypertext

MARK BERNSTEIN
Eastgate Systems
bernstein@eastgate.com

Hypertext and the World Wide Web are built on a complex network of ideas that originate in the military and ecological disasters of the short 20th Century, 1914-1989, and in a world-spanning intellectual effort to prevent their recurrence. Though that effort now appears to have failed, and though the tragic story of the 20th Century has not, it seems, concluded, we may learn profit by examining the concepts and ideals on which hypertext was built.

DOI: 10.1145/3545196.3545198 http://doi.acm.org/10.1145/3545196.3545198

The Web of 2022 is very different from (and for many, decidedly inferior to) the Web of 2002. It is not enough that we work to decrease the labor costs and increase the profit margins of Facebook and Twitter.

Hypertext and the World Wide Web were built on big ideas and understanding those ideas might help us rebuild them.

Any Questions / Comments / Doubts / Concerns?

Take-Home

- **Architectural constraints** are chosen to reach a desired **non-functional profile**: Lego, Gothic cathedrals, Bologna, Unix, networking
- Every constraint induces **trade-offs**: uniform interfaces gain **interoperability** and lose **design freedom**
- Which hypertext design is «better» **depends on the targeted non-functional requirements**; the Web succeeded with a simpler model than Xanadu
- **REST** combines **six constraints**: client-server, statelessness, cacheability, layered system, code-on-demand (optional), uniform interface
- The **uniform interface**: identifiers, manipulation via representations, self-descriptive messages, hypermedia

